

Article

Integrating Temporal Event Prediction and Large Language Models for Automatic Commentary Generation in Video Games

Xuanyu Sheng ¹, Aihe Yu ², Mingfeng Zhang ¹, Gayoung An ¹, Jisun Park ³ and Kyungeun Cho ^{4,*}

¹ Department of Computer Science and Artificial Intelligence, Dongguk University-Seoul, 30 Pildongro 1-gil, Jung-gu, Seoul 04620, Republic of Korea; shengxuanyu@dgu.ac.kr (X.S.); mingfeng@dgu.ac.kr (M.Z.); joaga0@dgu.ac.kr (G.A.)

² Department of Autonomous Things Intelligence, Dongguk University-Seoul, 30 Pildongro 1-gil, Jung-gu, Seoul 04620, Republic of Korea; aihe@dgu.ac.kr

³ NUI/NUX Platform Research Center, Dongguk University-Seoul, 30 Pildongro 1-gil, Jung-gu, Seoul 04620, Republic of Korea; jisun@dongguk.edu

⁴ Department of Computer Science and Artificial Intelligence, College of Advanced Convergence Engineering, Dongguk University-Seoul, 30 Pildongro 1-gil, Jung-gu, Seoul 04620, Republic of Korea

* Correspondence: cke@dongguk.edu

Abstract

Game commentary enhances viewer immersion and understanding, particularly in football video games, where dynamic gameplay offers ideal conditions for automated commentary. The existing methods often rely on predefined templates and game state inputs combined with an LLM, such as GPT-3.5. However, they frequently suffer from repetitive phrasing and delayed responses. Recent studies have attempted to mitigate the response delays by employing traditional machine learning models, such as SVM and ANN, for event prediction. Nonetheless, these models fail to capture the temporal dependencies in gameplay sequences, thereby limiting their predictive performance. To address these limitations, an integrated framework is proposed, combining a lightweight convolutional model with multi-scale temporal filters (OS-CNN) for real-time event prediction and an open-source LLM (LLaMA 3.3) for dynamic commentary generation. Our method incorporates prompt engineering techniques by embedding predicted events into contextualized instruction templates, which enables the LLM to produce fluent and diverse commentary tailored to ongoing gameplay. Evaluated in the Google Research Football environment, the proposed method achieved an F1-score of 0.7470 in the balanced setting, closely matching the best-performing GRU model (0.7547) while outperforming SVM (0.5271) and Transformer (0.7344). In the more realistic Balanced–Imbalanced setting, it attained the highest F1-score of 0.8503, substantially exceeding SVM (0.4708), GRU (0.7376), and Transformer (0.5085). Additionally, it enhances the lexical diversity (Distinct-2: +32.1%) and reduces the phrase repetition by 42.3% (Self-BLEU), compared with template-based generation. These results demonstrate the effectiveness of our approach in generating context-aware, low-latency, and natural commentary suitable for real-time deployment in football video games.

Keywords: video games; prompt engineering; large language models; time-series prediction model; game AI; machine learning

MSC: 68T05; 68T07; 62M10



Academic Editors: Ashutosh Mishra and Shodhan Rao

Received: 11 July 2025

Revised: 14 August 2025

Accepted: 19 August 2025

Published: 26 August 2025

Citation: Sheng, X.; Yu, A.; Zhang, M.; An, G.; Park, J.; Cho, K. Integrating Temporal Event Prediction and Large Language Models for Automatic Commentary Generation in Video Games. *Mathematics* **2025**, *13*, 2738. <https://doi.org/10.3390/math13172738>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In recent years, the role of commentary in digital games has shifted from a supplementary feature to a core component that significantly influences the experiences of both players and spectators. Particularly in sports simulation games and e-sports broadcasting, commentary plays a critical role not only in delivering real-time information, but in providing tactical analysis, emotional engagement, and immersive viewing experiences. High-quality game commentary facilitates an intuitive understanding of gameplay, lowers entry barriers for new audiences, and enhances both the commercial value and cultural impact of digital game content [1].

Despite their importance, live game commentaries predominantly depend on professional commentators in most scenarios. This reliance has created major challenges in terms of scalability, cost, language adaptability, and cross-cultural accessibility. To address these limitations, researchers have explored automated commentary systems that leverage advancements in NLP and LLM [2,3]. These systems have the potential to deliver real-time, context-aware, scalable, and personalized commentaries across a wide range of applications [4,5].

However, building an effective automatic commentary generator requires more than grammatically correct sentences. It requires the ability to perceive complex in-game dynamics, to understand the temporal context, and to generate fluent, diverse, and synchronized commentary that is aligned with the live gameplay. A particularly underexplored challenge is the real-time generation of diversified event commentary. Many existing methods rely on rule-based template systems to generate post hoc commentary, which often results in repetitive and inflexible outputs and fails to meet the requirements of real-time commentary [6]. Although some studies have explored the use of traditional machine learning models [7], such as SVM and ANN, to address commentary latency through event prediction, these approaches generally struggle to capture the temporal dependencies and dynamic evolution of game states, leading to limited predictive performance.

To overcome these limitations, a hybrid framework combines a lightweight time-series convolutional neural network—an omni-scale convolutional neural network (OS-CNN)—for high-precision real-time event prediction, together with LLaMA 3.3, a state-of-the-art open-source LLM enhanced through advanced prompt engineering methods [8]. OS-CNN is designed to capture both short- and long-range temporal dependencies through multiscale convolutional filters, enabling efficient and accurate sequence modeling. Prompt engineering, in this context, refers to the strategic design of input prompts to guide the LLM to generate contextually relevant, fluent, and emotionally rich outputs. By predicting game events and generating a corresponding commentary ahead of time, the proposed system significantly reduces response latency while improving linguistic diversity, contextual relevance, and naturalness. The main contributions of this study are as follows:

1. **A hybrid architecture for automated soccer commentary generation**
A hybrid framework specifically designed for soccer games combines time-series event prediction with natural language generation. This framework enables the automatic generation of event-driven commentaries, reducing the reliance on manual annotations and scripted templates.
2. **Real-time commentary generation based on three-second-ahead event prediction**
By leveraging a temporal prediction model to anticipate events three seconds in advance, the system enables real-time commentary generation with strong contextual consistency and responsiveness. In contrast to template-based methods that retrospectively generate commentary, this approach significantly reduces commentary latency, enhances the diversity and fluency of the output, and alleviates the repetitiveness typically associated with template-based generation.

The remainder of this paper is structured as follows. Section 2 reviews the related work on automatic commentary generation, event prediction models, and balanced training strategies. Section 3 describes the proposed framework, including the OS-CNN event predictor and the prompt-based language generation module. Section 4 presents the experimental setup and results, addressing both prediction accuracy and commentary quality metrics. Finally, Section 5 concludes the paper and outlines directions for future research.

2. Related Works

2.1. Research on Game Commentary Generation

Automatic commentary generation has attracted growing interest across various gaming genres, including racing games, fighting games, and e-sports. A common approach involves the use of an LLM, such as GPT-3.5 or ChatGPT, in combination with predefined templates. For instance, Nimpattavong et al. [9] applied ChatGPT to generate commentary for fighting games without fine-tuning the model, demonstrating its generalizability, albeit with a heavy reliance on rigid templates. Similarly, Renella and Eger [10] integrated EfficientDet-based event recognition with ChatGPT for League of Legends commentary, showcasing the potential for multimodal fusion between visual and textual inputs.

An alternative line of research has explored sequence-aware neural networks, such as Seq2Seq, LSTM, and transformer architectures. Wang and Yoshinaga [11] utilized game APIs and a Seq2Seq model to generate commentary for e-sports, whereas Li et al. [12] proposed a multimodal LSTM that focused on processing visual, motion, and audio features in platformer games. While these methods improved contextual fluency and expression diversity, they are mostly confined to offline or short-clip settings and lack the temporal resolution required for real-time deployment.

By contrast, domain-specific rule-based systems have been developed for games such as shogi and chess. Kameko et al. [13] applied rule-based parsing and external knowledge graphs for xiangqi commentary, and Zang et al. [14] used a CNN-based neural chess engine to generate well-structured descriptions. These systems achieve high semantic consistency but are limited in their expressive flexibility and responsiveness.

Overall, most existing methods follow a post hoc generation strategy, producing commentary only after events occur, which leads to delays, redundancy, and a lack of spontaneity [15,16]. Template-based methods suffer from repetitive phrasing [17], whereas multimodal LLM approaches are often hindered by increased noise and reduced textual accuracy [18]. These limitations highlight the importance of building a system that can predict events in advance and understand how the game changes over time, so it can generate real-time commentary more effectively. Table 1 provides a comparative overview of existing methods, highlighting key differences in event detection and generation strategies across various game genres, and demonstrates that most approaches still fall short of achieving real-time commentary.

Table 1. Research on Game Commentary Generation.

Ref.	Environment	Event Detection Method	Commentary Generation Method	Evaluation Method	Real-Time Commentary
Ishigaki et al. [15]	Racing Game	Softmax	LSTM	Auto + Human	×
Ishigaki et al. [17]		API	BART	Human	✓
Renella and Eger [10]	League of Legends	EfficientDet	ChatGPT	Auto + Human	✓
Wang and Yoshinaga [11]		API	Seq2Seq	Auto + Human	×
Nimpattavong et al. [9]	Fighting Game	API	ChatGPT	Auto + Human	×

Table 1. *Cont.*

Ref.	Environment	Event Detection Method	Commentary Generation Method	Evaluation Method	Real-Time Commentary
Kameko et al. [13]	Shogi	Rule-based Parsing + External Knowledge Grounding	Log-linear Language Model	Auto + Human	×
Mamoru et al. [16]	FPS	HUD Parsing	T5 Transformer	Auto + Human	×
Li et al. [12]	Platformer	LSTM	Seq2Seq + Attention	Auto	×
Zang et al. [14]	Chess	Neural Chess Engine (CNN)	LSTM + Attention	Auto + Human	×

2.2. Challenges in Soccer Commentary Generation

This study adopted Google Research Football (GRF) [19] as the simulation environment because of its close alignment with real-world soccer in terms of the rules and the physics. GRF offers frame-level high-resolution game states and a wide variety of event types, enabling the effective training and evaluation of prediction-based commentary systems. Moreover, its reproducibility and scalability facilitates data collection and repeated experimentation.

Table 2 summarizes the recent approaches to commentary generation in football games, including event detection methods, commentary generation models, evaluation protocols, real-time capabilities, commentary delays, and hybrid model designs. Existing methods predominantly rely on API-based event detection combined with an LLM, such as GPT-3.5 and LLaVA v1.5, for commentary generation. For instance, [6] employs GPT-3.5 with an API-based event detection, and combines automatic and manual evaluation; however, this approach does not support real-time commentary and still suffers from commentary delays. Similarly, [18] used LLaVA v1.5 entirely under automatic evaluation, but real-time commentary remained unsupported, and delays persisted.

Table 2. Research on Commentary Generation in Football Games.

Ref.	Environment	Event Detection Method	Commentary Generation Method	Evaluation Method	Real-Time Commentary	Commentary Delay	Hybrid Model
Czaplicki [6]	Google Research Football [19]	API	GPT-3.5	Auto + Human	×	✓	×
Stournaras [18]		API	LLaVA-v1.5	Auto	×	✓	×
Proposed Approach		OS-CNN	LLaMa3.3	Auto	✓	×	✓

Several recent studies have focused on enhancing the timeliness of commentaries by incorporating event prediction mechanisms. Kościółek [7] implemented machine learning models, such as SVM and ANN, to forecast game events and to generate commentary in advance. However, traditional models have inherent limitations. The SVM model assumes independence among features and fails to capture temporal dependencies [20–22]. Furthermore, the ANN model lacks the capability to model long-range dependencies and is prone to overfitting and information loss during the training and inference processes [23,24].

By contrast, our proposed method integrates an OS-CNN time-series model for event prediction with LLaMA 3.3 for commentary generation. Under a fully automated evaluation pipeline, our system successfully achieved real-time commentary generation, eliminated commentary delays, and incorporated a hybrid framework that enhanced prediction robustness and linguistic diversity.

Soccer gameplay data exhibit several essential characteristics that challenge conventional models. First, the data are sequential; player positions and ball trajectories evolve over time and encode predictive signals for upcoming events. For example, narrowing the defender spacing and increasing the attacker speed can indicate an imminent shot or pass. Second, the data are highly dynamic, with abrupt velocity shifts or directional changes leading to complex events, such as dribbles or tackles. Third, soccer data are multivariate and encompass 3D ball positions, relative velocities, fatigue levels, and tactical roles. Effective modeling of such interactions requires systems that can exploit multivariate temporal dependencies.

Conventional models, such as SVM and ANN, lack the capacity to process this structure, making time-series models a more appropriate choice. These models learn both short-term fluctuations and long-term trends, thereby enabling accurate real-time event prediction in complex environments.

2.3. Time-Series Models for Event Prediction

Recent advances in deep learning have indicated that time-series models are particularly well-suited for dynamic and sequential domains, such as soccer. Unlike static classifiers, time series models can capture temporal dependencies and evolve contextual patterns more effectively. These models can account for both short- and long-term relationships within the data, enabling a more robust understanding of temporally correlated events [25].

Representative architectures include TCN, RNN, GRU, and transformer-based models. The GRU network is widely employed to model long-range dependencies through gated memory mechanisms that help preserve contextual information over time. By contrast, transformer models leverage self-attention mechanisms and have shown strong performance in multiagent and multitask scenarios, making them particularly suitable for complex event prediction in sequential environments [26–28].

Overall, these models provide a foundation for efficiently processing temporally dynamic data and have been increasingly adopted for real-time decision-making tasks, including automatic commentary generation in sports applications.

Various examples of time-series-based deep-learning architecture have emerged for classification, forecasting, and anomaly detection. TimesNet [29] leverages 2D variation modeling to unify multiple time-series tasks, whereas the non-stationary transformer [30] handles shifting temporal distributions to improve prediction robustness. TS-CF [31], a deep forest model, achieves high accuracy through multilevel temporal features but suffers from inference latency and structural complexity. TimeMIL [32] introduced attention-based mechanisms to capture temporal relevance but may require extensive parameter tuning.

Compared with these, the OS-CNN model [33] used in this study offers several advantages:

- It uses multiscale convolutional kernels to simultaneously capture both short- and long-range temporal features.
- The CNN-based architecture is lightweight and fast, making it suitable for real-time applications.
- It effectively captures the flow of event transitions across time, which is particularly useful for modeling complex gameplay sequences.

Overall, the OS-CNN model balances temporal expressiveness, computational efficiency, and real-time applicability, making it well-suited for soccer commentary generation tasks.

While transformer-based models offer state-of-the-art performance in modeling global temporal dependencies, they typically necessitate large amounts of data and incur high

computational costs and inference latency. These factors pose significant limitations for real-time systems, such as automated commentary generation.

To clarify the motivation of our model choice, Table 3 presents a comparative overview of the popular temporal architecture.

Table 3. Comparative characteristics of the major temporal modeling architecture.

Model Type	Long-Term Dependency Modeling	Inference Efficiency	Model Complexity	Real-Time Suitability	Key Feature
RNN	Moderate	Low	Low	×	Recurrent structure
GRU	Good	Low	Medium	×	Gated memory cells
Transformer	Excellent	Low	High	×	Self-attention
OS-CNN	Good (multi-scale receptive fields)	High	Low	✓	Multi-scale CNN hierarchy

As shown, OS-CNN achieves a favorable trade-off by maintaining high inference speed and representational power with significantly fewer parameters.

Notably, our framework is modular and can be readily extended to incorporate Transformer-based architecture (e.g., TimeSformer, Informer) in future iterations, when real-time constraints are relaxed or more computational resources are available.

2.4. Importance of Balanced Datasets

This issue has been widely discussed in prior studies [34–38]. Event distribution in soccer games is inherently unbalanced. Critical gameplay events such as goals, fouls, and throw-ins occur far less frequently than background or “no-event” frames. This imbalance often leads models to overfit the majority classes during training, resulting in degraded performance on rare but semantically important events. Such bias not only limits the practical utility of commentary systems but also impairs their generalizability across diverse match scenarios.

Extensive prior research has demonstrated that balanced datasets can significantly improve model performance across multiple dimensions. For example, Narin et al. [34] and Valerde et al. [36] reported that balancing datasets improved recall for minority classes by more than 10% and reduced prediction bias, particularly in multi-class classification settings. Balanced training has also been shown to enhance MCC, F1, and AUC metrics, contributing to improved model stability, reproducibility, and evaluation reliability [35,37,38]. Moreover, balanced datasets promote classifier fairness and generalization, independent of the learning algorithm used. Their effectiveness has been validated across various models, including SVM, ANN, CNN, and XGBoost. Balanced data also helps prevent overfitting in rare-event prediction and supports more effective hyperparameter tuning under test-time conditions.

To address the class imbalance issue, this study adopts both down-sampling and up-sampling strategies. The down-sampling strategy involves randomly reducing the number of samples in majority classes to match those of minority classes, thereby preventing the model from being biased toward overrepresented categories. In addition, the up-sampling strategy employs the Synthetic Minority Over-sampling Technique (SMOTE), which generates synthetic samples for minority classes by interpolating between existing instances. Through these combined strategies, the model is able to learn each category more equally and to reduce bias toward the majority.

Specifically, for the dominant “no event” class, random down-sampling was applied to select instances matching the minority class size. Each selected sample involved a sequence of three consecutive frames ($t - 2$, $t - 1$, t) with no labeled event occurring at $t + 3$, ensuring

temporal continuity. This structure preserves the sequential nature of the input required by OS-CNN and enables the effective learning of contextual transitions.

By balancing the dataset, our approach not only mitigates the model's bias toward frequent "no-event" frames but improves its ability to detect meaningful transitions in gameplay. This strategy ultimately enhances the accuracy and reliability of the system in generating context-aware, real-time commentary.

2.5. Summary

Recent developments in automatic game commentary have led to significant progress across various game genres, including racing, fighting, e-sports, and sports simulation games. The existing methods can generally be categorized into three major groups: rule-based systems, template-driven approaches combined with LLM, and sequence-aware deep learning models.

Rule-based systems rely on predefined logical structures and external knowledge bases to generate structured and semantically consistent commentary, and are particularly effective for domain-specific games such as chess and shogi. However, these systems exhibit limited flexibility, lack adaptability to dynamic gameplay scenarios, and are incapable of effectively handling real-time generation.

Compared with rigid rule-based systems, template-driven approaches enhanced by large language models (such as GPT-3.5, ChatGPT, and LLaVA) improve linguistic diversity and fluency. These methods demonstrate a certain degree of generalization across different games without requiring model fine-tuning. Nevertheless, they are overly dependent on handcrafted prompt templates and suffer from rigid expression patterns, delayed responses, and lack of contextual synchronization with game events. Furthermore, these models are susceptible to noisy input signals and often struggle to maintain stable textual accuracy during dynamic gameplay.

Sequence-aware deep-learning models, including Seq2Seq, GRU, and transformer, among other time-series models, have been designed to capture temporal dependencies and continuously evolving game states. These models excel at modeling complex temporal patterns and generating a more coherent and fluent commentary. However, most existing sequence-based methods operate in retrospective or offline settings and cannot anticipate future events. As a result, they cannot support low-latency commentary aligned with upcoming gameplay actions, which limits their suitability for real-time applications.

Soccer gameplay presents unique challenges owing to its highly dynamic, sequential, and multivariate nature. Critical game events, such as goals, fouls, and throw-ins, are inherently rare compared to the abundant "no event" background frames, resulting in severe class imbalance problems. Without proper data balancing, models often overfit the majority classes and degrade their performance on rare but semantically important events. Extensive research has demonstrated that balanced datasets can improve minority class recall, reduce prediction bias, and enhance model stability, reproducibility, and evaluation reliability across various models.

To bridge the gaps in the existing methods—namely, lack of anticipation, limited real-time capability, poor handling of class imbalance, and repetitive language generation—we propose a hybrid framework that integrates a lightweight time-series convolutional model (OS-CNN) with improved prompt-engineered large language models (LLaMA 3.3). Supported by balanced data sampling, this framework effectively captures both short- and long-term temporal features while mitigating class imbalance. Predicting events in advance and dynamically generating context-aware commentary, it enables real-time, fluent, and expressive outputs that better align with gameplay dynamics.

3. Proposed Method

3.1. Overview

To achieve real-time automatic commentary generation for soccer games, a three-stage integrated framework is proposed, as illustrated in Figure 1. The system operates in a pipeline across sequential time windows to ensure real-time processing and output.

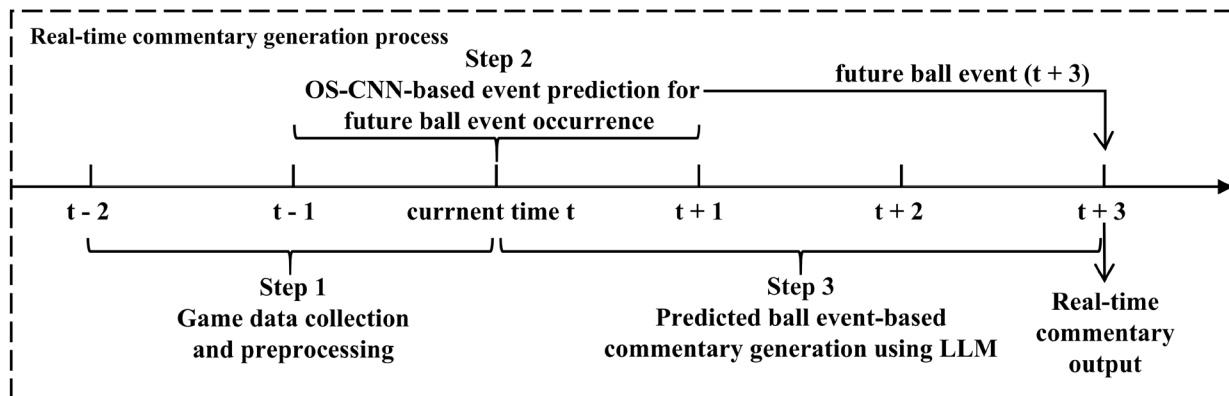


Figure 1. Full process of real-time automatic commentary generation system. This figure depicts the proposed three-stage integrated framework, from game data collection and preprocessing (from $t - 2$ to t) to the OS-CNN model predicting future events (at $t + 3$). Subsequently, the LLM generates real-time commentary based on these anticipated events to ensure synchronization with the ongoing gameplay.

In Step 1, the system continuously collects and preprocesses game data from time steps $t - 2$ and $t - 1$ up to the current time t , preparing structured input features for subsequent analysis.

In Step 2, an OS-CNN-based event prediction module analyzes the accumulated data within the window, from $t - 2$ to t , to predict the most probable future ball event occurring at time $t + 3$.

Finally, in Step 3, the predicted future ball event, along with the relevant game context, is fed into an LLM to generate commentary text describing the predicted event. The generated commentary is then delivered at time $t + 3$, ensuring that both real-time responsiveness and high-quality commentary output are synchronized with the predicted event occurrence.

The decision to predict events with a 3s offset (i.e., generating commentary for events predicted to occur at $t + 3$) is based on the empirical design proposed by Kościółek [7], who exhibited that aligning gameplay data three seconds before an event yields high predictive accuracy and maintains real-time applicability in commentary systems. Following this precedent, the event prediction framework processes data within the time window from $t - 2$ to t and forecasts the most probable ball event at $t + 3$. This design facilitates real-time responsiveness while ensuring methodological consistency with previously validated approaches. Future work may investigate whether different event types, such as corner kicks or goal kicks, could benefit from adaptive lead times based on their unique temporal characteristics observed in real-world matches.

The proposed system integrates real-time event anticipation, multiframe contextual feature extraction, and natural language generation for automatic soccer commentaries, as illustrated in Figure 2. The architecture comprises three stages, forming a continuous pipeline from game data acquisition to real-time commentary delivery.

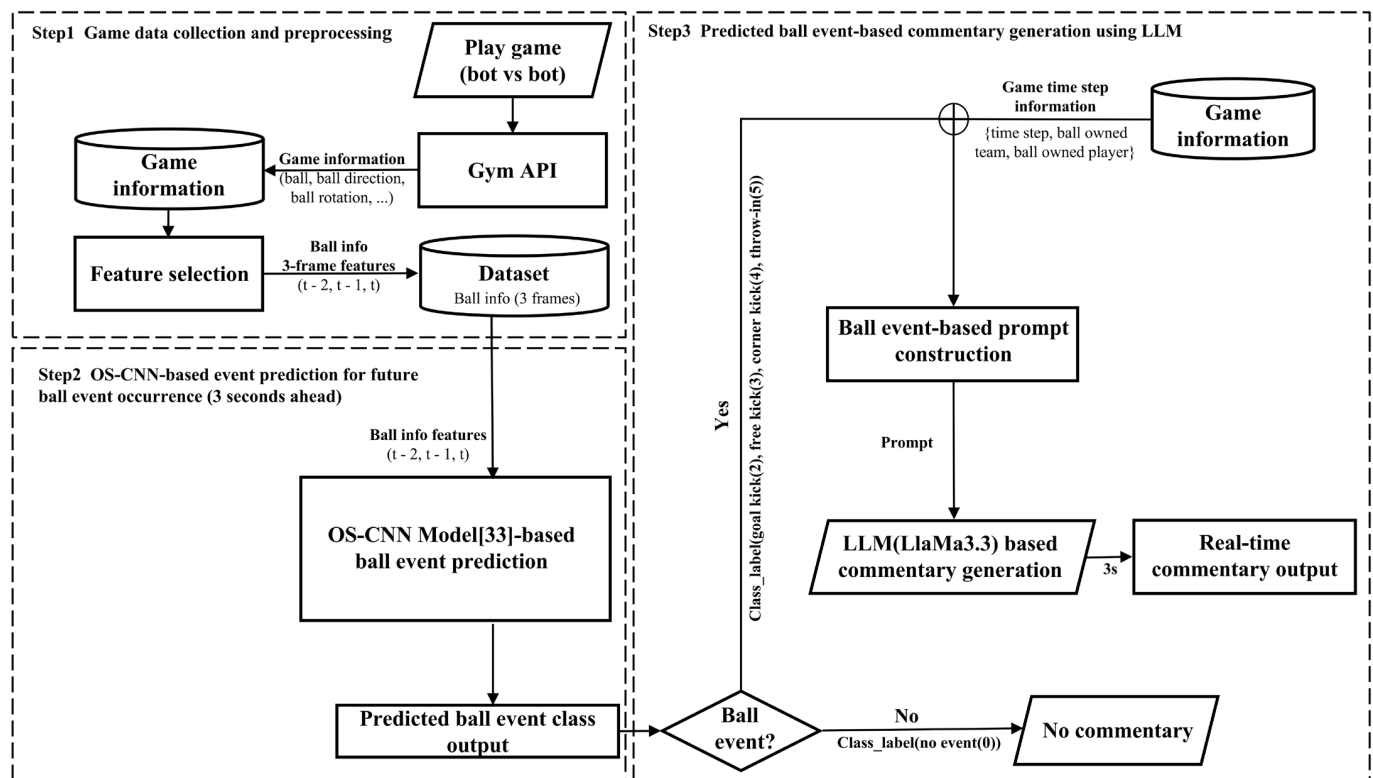


Figure 2. Full architecture of the real-time automatic commentary generation system. This figure presents the detailed architecture of the proposed system, demonstrating the three interconnected stages: continuous gameplay data acquisition and preprocessing from Google Research Football, real-time future event prediction by the OS-CNN model, and dynamic, context-aware commentary generation by LLaMA 3.3 via prompt engineering for immediate output.

In Step 1, the gameplay data were continuously generated through bot-versus-bot simulations using the play game (bot vs. bot) module via the gym API. The game information module extracts detailed ball-related attributes, including ball position, direction, rotation, and other relevant features. These raw features were processed through a feature selection component to filter and transform them into structured inputs, focusing on 3-frame ball information sequences ($t - 2$, $t - 1$, t). The dataset module organizes these multi-frame features as inputs for event prediction.

In Step 2, the extracted ball information features are fed into the OS-CNN model [33], which consists of stacked OS blocks, multiscale convolutions, BatchNorm + ReLU activations, and concatenation layers. This hierarchical architecture effectively captures short-term temporal dynamics and contextual dependencies. After feature extraction, the outputs were aggregated via global average pooling and mapped to event classes through a fully connected layer, yielding a predicted ball event class output (`class_label`). The system predicts imminent ball-related events—goal kick (2), free kick (3), corner kick (4), and throw-in (5)—occurring 3 s ahead, enabling proactive commentary.

In Step 3, if the predicted class label indicates an upcoming event (`class_label`: 2, 3, 4, or 5), the system triggers the ball event-based prompt construction module. This module combines the predicted event type with contextual game information (time step, ball-owned team, and ball-owned player) to construct a customized prompt. The generated prompt is then passed to the LLM (LLaMA 3.3) for natural language generation. The LLaMA 3.3 model was chosen for its strong language generation capabilities, multilingual support, and efficient inference compared to other open-source LLMs, such as BLOOMZ or Falcon. While models like BLOOMZ offer comparable performance, LLaMA 3.3 demonstrated

better latency and coherence under prompt-tuned inference in preliminary experiments. Additionally, its robust instruction-following behavior made it suitable for integrating structured event data into commentary generation without fine-tuning. The resulting commentary is output through the real-time commentary output module, with a three-second lead time to align with the anticipated event occurrence. If the predicted class is no event (class_label: 0), the process follows the No Commentary branch, skipping the commentary generation for that time step.

Compared with conventional template-based methods, this integrated architecture enhances narrative diversity and responsiveness, generating fluent and context-aware commentary while minimizing redundancy.

3.2. Game Data Collection and Preprocessing

To train the event prediction module, game data were collected from GRF [19], which is a physics-based soccer simulation environment. In Step 1, the gameplay data are autonomously generated through bot-versus-bot simulations using the play game (bot vs. bot) module under the gym API interface. During each simulation, the game information module continuously recorded the game state information at every frame, capturing a total of 481 raw features per frame, including the ball position (ball_x, ball_y, ball_z), ball movement direction (ball_direction_x, ball_direction_y), ball rotation, and other gameplay states.

Following raw data extraction, the system performs a three-second temporal window extraction to create structured input sequences. Specifically, the three-second temporal window extraction module segments the game data into three-frame temporal windows, corresponding to frames $t - 2$, $t - 1$, and t , resulting in an intermediate feature sequence with shape $(t - 2, t - 1, t) \times 481$.

Subsequently, feature dimensionality reduction was performed using the feature-selection component, which selected 123 key ball-related features from the original 481 features for each frame. This step yields ball information features with dimensions $(t - 2, t - 1, t) \times 123$, effectively retaining informative signals while filtering out redundant or irrelevant data.

Finally, the selected features were organized into a structured dataset using the dataset module. Thus, each data instance consisted of three consecutive frames (3×123 features) capturing ball-state information across time, ready for input into the event prediction model. A three-frame temporal window $(t - 2, t - 1, t)$ was adopted to balance prediction accuracy and real-time responsiveness. This configuration offers short-term temporal context sufficient to capture event transitions while minimizing latency and noise, which are critical factors in real-time applications. The effectiveness of short fixed-length windows has also been exhibited in prior work [7]. The complete data-collection and preprocessing pipeline is illustrated in Figure 3, summarized step-by-step in Table A7, and the full pseudocode is provided in Appendix C.

Although the simulated environment allows for controlled data generation and consistent labeling for event prediction, it does not fully capture the complexity and variability inherent in real-world soccer matches. Professional match commentary includes richer contextual understanding, spontaneous phrasing, and diverse linguistic styles. As part of future work, real-world broadcast footage and human-written commentaries will be integrated to improve the naturalness and robustness of the system under authentic gameplay conditions.

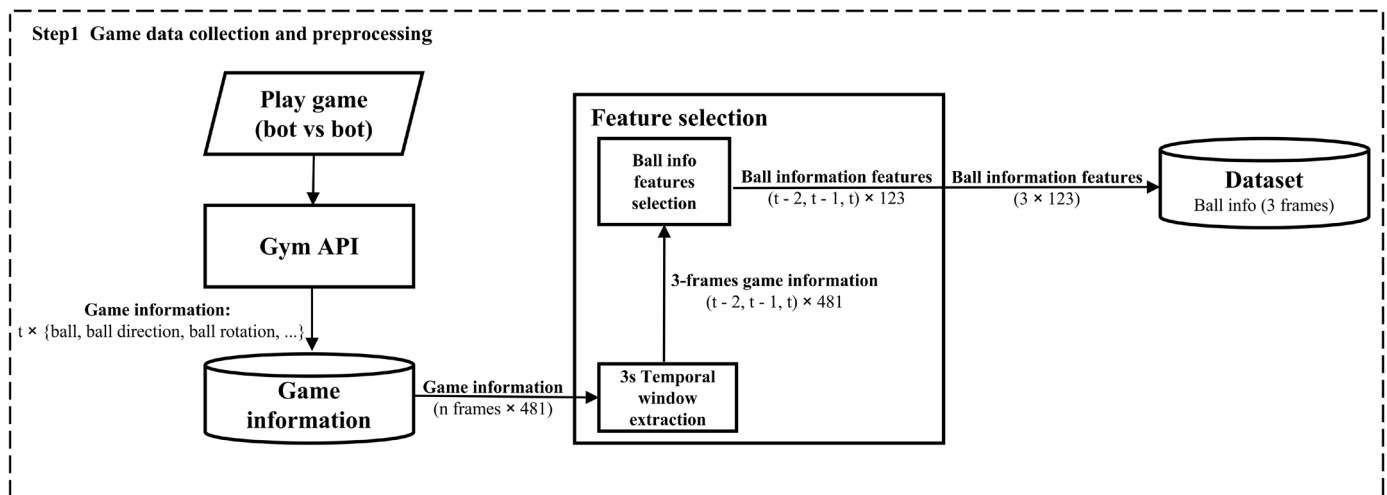


Figure 3. Game data collection and preprocessing pipeline. This figure depicts the process of obtaining raw game state data from the Google Research Football environment, extracting a refined set of 123 ball-related features, and structuring them into three-frame temporal windows ($t - 2, t - 1, t$) for subsequent event prediction.

3.3. OS-CNN-Based Event Prediction for Future Event

For future ball event prediction, an OS-CNN model [33] was employed to classify upcoming soccer events 3 s in advance based on ball information features. As illustrated in Figure 4, the input to the OS-CNN model consists of ball information features with dimensions (3×123) , where 3 indicates the temporal length (3 consecutive frames) and 123 represents the number of features extracted per frame.

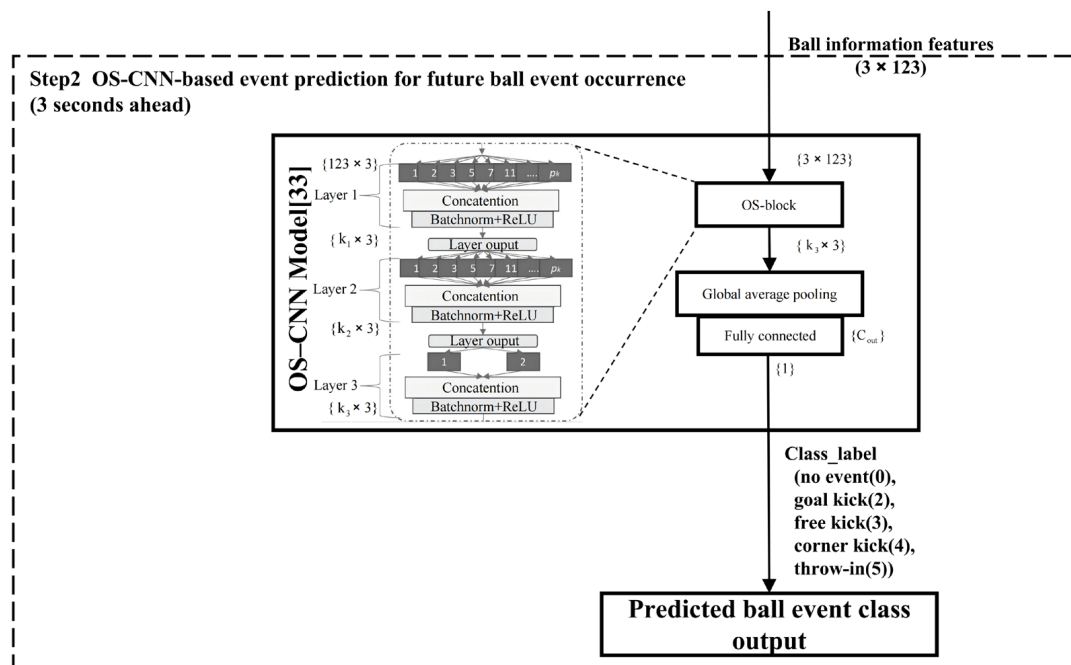


Figure 4. Architecture of the OS-CNN-based event prediction module. This figure details the structure of the OS-CNN model, showing how the model processes 3×123 ball-information features through multiple omni-scale blocks with multi-scale convolutional filters to capture temporal dependencies, followed by global average pooling and a fully connected layer with softmax activation to predict one of five ball event categories.

The OS block module serves as the core of the architecture. It applies multiple one-dimensional convolutional filters with diverse kernel sizes (k_1, k_2, k_3) in parallel to capture multi-scale temporal dependencies. Each OS block layer includes the following sequential operations:

1. Convolution operations using kernels of various sizes to extract features at multiple temporal scales.
2. Concatenation of outputs from all kernel branches to merge the multi-scale features.
3. Batch normalization to stabilize learning and to improve convergence.
4. ReLU activation to introduce non-linearity.

This processing pipeline is repeated hierarchically across the three layers (Layers 1, 2, and 3), progressively refining temporal representations. The output shape at each layer remains $\{k_i \times 3\}$, preserving temporal resolution while expanding feature abstraction.

After processing through the final OS block layer, the extracted feature maps are passed through a global average pooling layer that compresses the temporal dimension and produces a compact feature vector of size (C_{out}). This vector is then fed into a fully connected (FC) layer, projecting it into class logits.

Finally, a softmax activation function was applied to generate class probabilities, predicting the ball-event class output among five possible categories: no event (0), goal kick (2), free kick (3), corner kick (4), and throw-in (5).

This OS-CNN-based architecture efficiently captures both short- and long-term temporal patterns in ball dynamics through its multiscale convolutional design. Its lightweight structure ensures real-time inference suitability for soccer event prediction tasks. The detailed architecture and data flow of the OS-CNN-based event prediction module are illustrated in Figure 4.

3.4. Event-Prediction-Based Commentary Generation Using LLMs

The third stage of the framework focuses on generating fluent and context-aware commentary on predicted ball events using LLMs. As illustrated in Figure 5, the pipeline consists of the following sequential components:

1. **Event Triggering:** The process begins with a ball event. The decision module checks whether the predicted ball event class label indicates a valid event (i.e., goal kick, free kick, corner kick, or throw-in with class labels $\in \{2, 3, 4, 5\}$). If a valid event is detected, the pipeline proceeds to a prompt construction stage. Otherwise, if the class label is zero (no event), the system bypasses commentary generation to conserve computational resources.
2. **Prompt Construction:** When an event is triggered, the game information module provides essential game time-step information, including the time step, ball-owned team, and ball-owned player information. These details, combined with the ball event type from the event prediction output, are used to construct a prompt following a predefined template. The prompt template incorporates structured placeholders as follows:
 - {time_event}: Current time step of the event;
 - {event_type}: Type of ball event (e.g., goal kick);
 - {team}: Ball-owned team;
 - {ball owned player}: Player currently possessing a ball;
 - {benefiting_team}: Teams benefiting from the event.
 - This template ensures the generated commentary is informative, situationally grounded, and expressive.

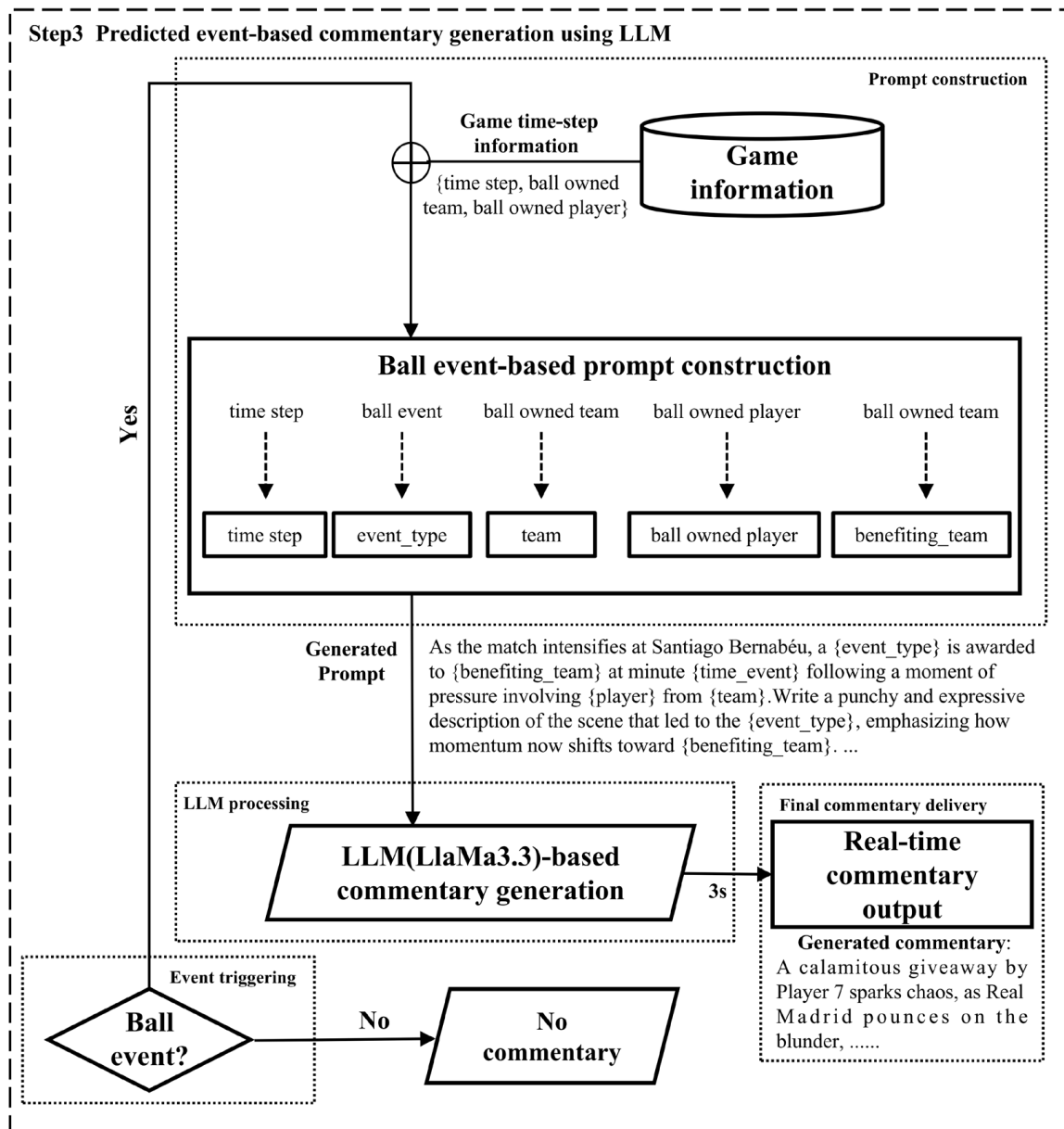


Figure 5. Pipeline of commentary generation process. This figure illustrates the three-step pipeline for commentary generation, starting with event triggering based on predictions of the OS-CNN model, followed by the dynamic construction of context-aware prompts using game information, and finally, the generation of fluent and synchronized commentary by LLaMA 3.3 for real-time delivery, while bypassing generation for ‘no event’ states.

To support commentary generation, two sets of prompt templates were constructed: one for goal-related events and another for non-goal events. Each template incorporates the predicted event type, relevant contextual variables (e.g., player identity, game time, current score), and emotional indicators to guide the generation process. The prompts are designed to avoid repetitive expressions and to preserve linguistic variety while limiting each output to 75 words to ensure speed and fluency. Structured placeholders (e.g., {chosen_emotion}) allow for dynamic control over emotional tone based on match dynamics. A further evaluation of the prompt structure is presented in Section Improved Prompt Design.

3. **LLM Processing:** The constructed prompt is fed into the LLM (LLaMa3.3) module, which generates fluent, natural, and contextually relevant commentary text reflecting the predicted ball event and gameplay context. The generative capabilities of the

LLM enable diverse narrative expressions, overcoming the repetitiveness of purely template-based systems.

4. Final Commentary Delivery: The generated commentary output is then delivered via the real-time commentary output module, synchronized to maintain a three-second lead time relative to the predicted event occurrence. This ensures that viewers receive timely and natural narration aligned with live gameplay, enhancing immersion and maintaining a smooth narrative flow.

Overall, this commentary generation pipeline integrates event triggering, context-driven prompt construction, LLM-based generation, and real-time delivery to produce a dynamic, human-like match commentary for each detected event.

4. Experimental Setup and Results

4.1. Implementation Details

To validate the proposed framework, separate experiments were conducted for event prediction and commentary generation. All the experiments were conducted on an Ubuntu 22.04.3 LTS system equipped with an NVIDIA GeForce RTX 3090 GPU. The model training and inference were implemented using PyTorch 2.4.1, CUDA 11.8, and cuDNN 9. The LLaMA 3.3 large language model was used to generate the commentary text.

The dataset comprised 1247 simulated soccer matches. Data preprocessing required approximately 150 h. Training the event prediction model required approximately three days, whereas testing the commentary generation pipeline required approximately 3 h, demonstrating the framework's potential for real-time deployment, as summarized in Table 4.

Table 4. System Configuration and Runtime Summary.

Category	Info
Operating System	Ubuntu 22.04.3 LTS
GPU	NVIDIA GeForce RTX 3090
Library Stack	PyTorch 2.4.1, CUDA 11.8, cuDNN 9
Large Language Model	LLaMA 3.3
Dataset Processing Time	~150 h
Event Prediction Training Time	~3 days
Commentary Generation Test Time	~3 h

In addition to the system configuration, the OS-CNN hyperparameters for event prediction are summarized in Table 5. These include kernel sizes, filter counts, regularization components, and training details to facilitate reproducibility and further comparison. To address class imbalance, a class-weighted Focal Loss with clipped log-scaled weights was employed, and the SMOTE was applied to balance the training samples across event categories. These components ensure that both frequent and rare events are appropriately emphasized during training.

For the GRU and Transformer baselines, we conducted a grid search to optimize the key hyperparameters. The GRU model was tuned over hidden sizes (32, 64, 128), number of layers (1, 2), and learning rates (0.001, 0.0005), with the batch size fixed at 32. For the Transformer model, the number of heads (2, 4), feedforward dimensions (64, 128), and encoder layers (1, 2) were adjusted. All models were evaluated using five-fold stratified cross-validation, and the best configurations were selected based on the weighted F1-score. This procedure ensures a fair and consistent comparison across the architecture under well-optimized settings.

Table 5. OS-CNN Hyperparameter Configuration.

Component	Value/Description
Model Layers	3 OS-Blocks (each with multiple kernel branches), 1 final FC layer
Prime Kernel Sizes	Automatically generated from start_kernel_size = 1 to Max_kernel_size = 3
Filters per Kernel	Automatically determined via generate_layer_parameter_list() based on parameter budget
Layer Params	parameter_number_of_layer_list = [1*4] → roughly four parameters per channel
Activation	ReLU (after BatchNorm)
Dropout	0.3 (applied after ReLU)
Pooling Layer	AdaptiveAvgPool1D (output size = 1)
Loss Function	Class-Weighted Focal Loss ($\gamma = 2.0$), using log1p-scaled class weights clipped to [1.0, 10.0] to address severe class imbalance during event prediction. $L_{\text{focal}} = -\text{clip}(\log(1 + \frac{1}{f_c}), 1.0, 10.0) \cdot (1 - p_t)^\gamma \cdot \log(p_t)$
Optimizer	AdamW
Learning Rate	0.0005
Weight Decay	1×10^{-6}
Scheduler	CosineAnnealingLR ($T_{\text{max}} = 2000$, $\eta_{\text{min}} = 1 \times 10^{-5}$)
Regularization	L2 regularization added ($\lambda = 1 \times 10^{-4}$)
Gradient Clipping	max_norm = 5
Batch Size	Dynamic: min(samples/10, 16), minimum batch size = 2
Epochs	100
Data Sampling	SMOTE for class balancing
Early Stopping	Stop after five epochs with no F1 improvement
Pretraining	Supports loading pretrained weights if available (optional)

4.2. Dataset Description

4.2.1. Data Collection for Event Prediction

Two datasets were constructed to evaluate the robustness of the model under both ideal and realistic settings:

- **Balanced Dataset:** Down-sampled from 1247 matches to ensure equal class distribution (837 samples per class).
- **Imbalanced Dataset:** Constructed by randomly sampling 100 matches to obtain a naturally imbalanced dataset that reflects real-world game scenarios. During training, the SMOTE was applied to balance the dataset and to allow the model to fully learn the characteristics of each event type.

This dual-dataset strategy enables a comparative analysis of performance under class-balanced and real-world scenarios, providing insight into both the structural capability and the practical deployment feasibility. Detailed event distributions for both datasets are summarized in Table 6.

In the prediction experiments, rare classes, such as goal, kick-off, and penalty, were excluded in line with [7], limiting the prediction task to the five most frequent event types: no event, goal kick, free kick, corner kick, and throw-in.

These five events were selected not only because of their greater frequency in the dataset, but due to their structural clarity and relevance to real-time commentary generation. Each of these events represents a distinct and well-defined transition in gameplay that can be reliably anticipated within a short temporal window (3 s). By contrast, events such as goal, penalty, and kick-off were excluded because they are extremely rare and depend on longer-term contextual build-up or external refereeing conditions, which make them less suitable for short-horizon prediction. Selecting this subset of common, structurally consistent events ensures more stable training, fair evaluation, and comparability with prior work [7].

Table 6. Event Distribution in Dataset.

Dataset	Set Type	Matches	Event Type	Raw Event Count	Sampled Event Count
Balanced	Test	1247 matches	No event (0)	3,647,134	837
			Goal kick (2)	1658	837
			Free kick (3)	9025	837
			Corner kick (4)	837	837
			Throw-in (5)	2394	837
	Train		No event (0)	3,647,134	837
			Goal kick (2)	1658	837
			Free kick (3)	9025	837
			Corner kick (4)	837	837
			Throw-in (5)	2394	837
Imbalanced	Test	100 matches	No event (0)	291,255	291,255
			Goal kick (2)	133	291,255
			Free kick (3)	708	291,255
			Corner kick (4)	76	291,255
			Throw-in (5)	234	291,255
	Train		No event (0)	291,255	291,255
			Goal kick (2)	133	133
			Free kick (3)	708	708
			Corner kick (4)	76	76
			Throw-in (5)	234	234

4.2.2. Feature Selection and Preprocessing

The GRF environment offers 481 frame-level features. To reduce model complexity and to enhance computational efficiency, a subset of 123 features was selected based on prior work [7] and further refined to capture soccer-specific dynamics. These features encompass critical aspects of gameplay and player status, including ball status (position, direction, and rotation), team/player positions and directions, player fatigue levels, ball possession indicators, game mode, and score. Thus, each frame captures a rich semantic snapshot of the ongoing game, allowing the model to interpret both spatial and contextual information effectively.

Each training sample consisted of three consecutive frames preceding an event, forming a time-series input of shape (3, 123), where 3 represents the temporal dimension and 123 corresponds to the selected features. To ensure stability and comparability across the samples, the features were normalized using mean removal and unit variance. The temporal structure of the sequence is preserved to capture the short-term dynamics relevant to event prediction. Table 7 provides an overview of the types and quantities of information encoded in each frame.

Table 7. Information Description for Each Frame.

Name	Description	Number
Ball	Position: [x, y, z]	3
Ball Direction	Movement vector: [x, y, z]	3
Ball Rotation	Rotation (in radians): [x, y, z]	3
Ball Owned Team	Owning team	1
Ball Owned Player	Player index	1
Team	Positions: [x, y]	44
Team Direction	Movement vectors: [x, y]	44
Team Tired Factor	Fatigue levels (0 = energetic, 1 = exhausted)	22
Team Yellow Card	Yellow cards count	22
Team Active	Active status in game	22

Table 7. Cont.

Name	Description	Number
Team Roles	Roles (e.g., forward, defender)	22
reward	The immediate reward received at the current time step, reflecting outcomes such as goals or intermediate progress (e.g., checkpoints).	1
cumulative_reward	The accumulated reward over an episode or a given horizon, used to evaluate overall performance.	1
Designated	Designated player index (e.g., ball owner)	2
Sticky Actions	Current active actions (10-dimensional one-hot vector: move directions, sprint, dribble)	286
Score	Score (number of goals)	2
Steps Left	Remaining steps in match	1
Game Mode	Current game mode (e.g., no event, kick-off, goal kick, free kick, corner, throw-in, penalty)	1

4.3. Evaluation Metrics

To comprehensively evaluate the system performance, two groups of metrics were used, including (1) prediction metrics for assessing the accuracy and robustness of event classification, and (2) language generation metrics for evaluating the fluency, diversity, and responsiveness of the generated commentary.

4.3.1. Event Prediction Metrics

The performance of the event prediction model was evaluated using three standard classification metrics commonly used in machine learning: recall, precision, and F1-score. These metrics provide a comprehensive view of how well the model identifies and distinguishes between the various event classes.

- Recall (1):

Recall measures the ability of a model to correctly identify all relevant instances for a given class. It is defined as the proportion of true positives among all actual positive cases. A high recall indicates that most of the actual events are successfully detected by the model.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (1)$$

- Precision (2):

Precision evaluates the number of predicted events that are correct. It is calculated as the proportion of true positives among all the predicted positives. High precision indicates that the model has low false alarm rates and is reliable for identifying predicted events.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (2)$$

- F1-Score (3):

The F1-score is the harmonic mean of precision and recall. This provides a balanced metric when both the false positives and negatives are important. This is particularly useful in multiclass or imbalanced classification settings, such as soccer event prediction, where class distributions may vary significantly.

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3)$$

These metrics were calculated for both the balanced and imbalanced datasets to evaluate not only the overall predictive performance but the robustness of the model under

class distribution shifts. In this study, the F1-score was used as the primary metric for comparative evaluation, owing to its sensitivity to class imbalance.

4.3.2. Commentary Evaluation Metrics

Several natural language generation (NLG) metrics were used to evaluate the quality, diversity, and efficiency of the generated commentaries. These metrics included perplexity, distinct-n, lexical diversity, self-BLEU, and generation time.

Perplexity measures the uncertainty of a language model when predicting the next word in a given sequence. It is a widely used metric in natural language processing for evaluating the fluency and syntactic coherence of generated text, with lower perplexity values typically reflecting better predictive confidence and structural consistency. In traditional language-generation tasks, such as machine translation or formal text synthesis, minimizing perplexity is closely associated with higher linguistic quality.

However, in the domain of sports commentary, particularly real-time or spontaneous speech-like narration, a slightly elevated perplexity may actually be beneficial. Higher perplexity in this context often corresponds to more expressive, emotionally charged, or contextually varied outputs deviating from rigid grammatical norms. These variations are not indicative of poor generation quality but rather signal the presence of dynamic phrasing, rhetorical emphasis, or colloquial structures that enhance the realism and engagement of the commentary [39–41]. Therefore, perplexity should be interpreted in a context-sensitive manner, recognizing that expressiveness and spontaneity, essential qualities in sports commentary, may naturally lead to higher perplexity without sacrificing communicative effectiveness.

Distinct-n [42] evaluates the proportion of unique n-grams in the generated text. For instance, Distinct-1 measures the ratio of unique unigrams to total unigrams, whereas Distinct-2 measures bigrams. Higher values reflect increased lexical variation and reduced repetition, which are essential for avoiding monotony during repetitive game events.

Lexical diversity [43], typically quantified using the TTR, indicates the breadth of the vocabulary used in the generated text. A higher lexical diversity score suggests that the system produces a more varied and natural language, avoiding overly generic or formulaic phrasing.

Self-BLEU [44] assesses similarity among the generated sentences by treating each sentence as a hypothesis and the remaining sentences as references. A lower self-BLEU score implies higher inter-sentence diversity, which is crucial in sports commentary to ensure that similar events are described using varied expressions and phrasing, thereby maintaining the audience's interest.

Generation time refers to the average time (s) required to generate a single commentary sentence. Shorter generation times imply better suitability for real-time applications, such as live sports broadcasting, where low latency is critical.

Together, these metrics formed a comprehensive evaluation framework. Whereas prediction metrics assess the accuracy of event detection, language generation metrics ensure that the system produces a fluent, diverse, and time-efficient commentary suitable for real-time deployment. These metrics were reported and compared in the experiments to validate the classification and generation components of the proposed system.

4.4. Event Prediction Results

4.4.1. Confusion Matrix Analysis: Balanced Dataset

To evaluate the classification performance under balanced conditions, a five-fold cross-validation was conducted across five event classes. The proposed OS-CNN model achieved competitive and stable classification performance across all categories. As shown

in Table 8 and Figure 6d, it achieved strong performance on goal kick (F1 = 0.72), corner kick (F1 = 0.75), and throw-in (F1 = 0.92), indicating that the sequential convolutional structure effectively captures localized temporal transitions during gameplay. Notably, although its performance on the no-event class was lower (F1 = 0.64), it still outperformed SVM by a large margin (F1 = 0.39).

Table 8. Per-Class Evaluation Metrics (Balanced Dataset).

Method	Goal Kick (2)			Free Kick (3)			Corner Kick (4)			Throw-In (5)			No-Event (0)		
	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑
Kościełek [7]	0.54	0.53	0.53	0.50	0.49	0.49	0.56	0.59	0.58	0.63	0.65	0.64	0.4	0.38	0.39
GRU [45]	0.72	0.70	0.71	0.74	0.73	0.73	0.71	0.77	0.74	0.93	0.88	0.91	0.69	0.70	0.69
Transformer [46]	0.68	0.75	0.71	0.70	0.71	0.70	0.70	0.72	0.71	0.90	0.92	0.91	0.69	0.59	0.64
Proposed Approach	0.76	0.68	0.72	0.69	0.73	0.71	0.68	0.83	0.75	0.88	0.96	0.92	0.76	0.56	0.64

Note: Precision (P), Recall (R), and F1-score (F1). Bold values indicate the best performance in each column.

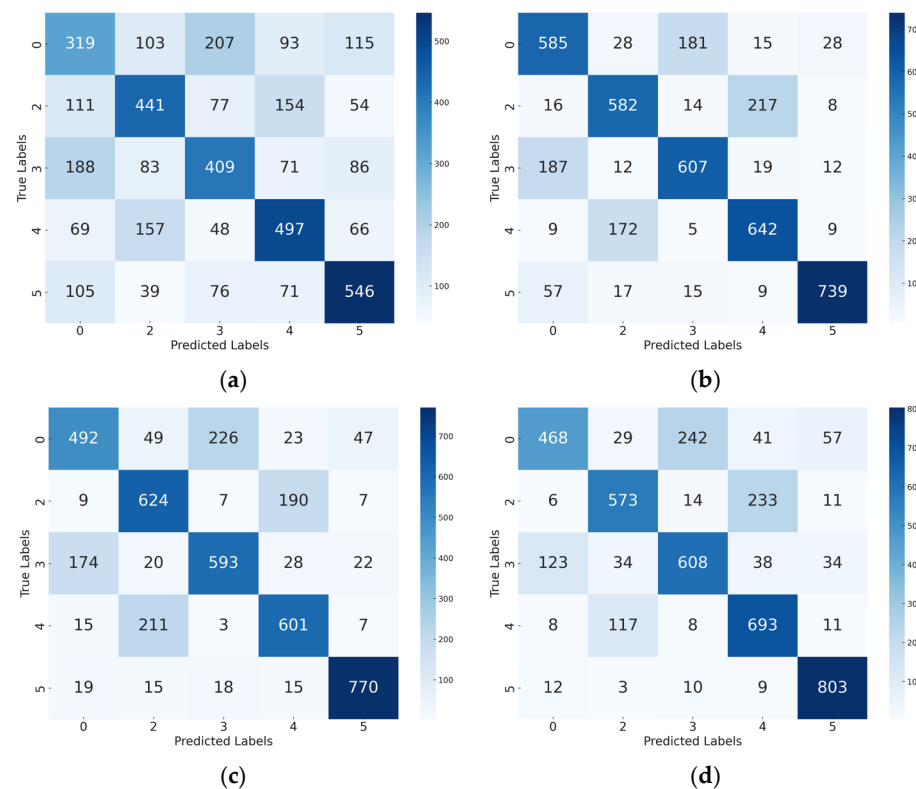


Figure 6. Confusion matrices for event prediction on the balanced Dataset: (a) confusion matrix for SVM (Balanced dataset); (b) confusion matrix for GRU (Balanced dataset); (c) confusion matrix for Transformer (Balanced dataset); (d) confusion matrix for OS-CNN (Balanced dataset).

Compared with OS-CNN, the RNN-based models, such as GRU, achieved slightly higher F1-scores for the throw-in class (0.91 and 0.92, respectively). The Transformer model displayed a generally stable performance across most classes, with F1 scores ranging from 0.64 to 0.71, but did not outperform the proposed approach on any individual category.

By contrast, the SVM baseline showed an overall weak performance across all classes, particularly on the no-event class (F1 = 0.39), goal kick (F1 = 0.53), and free kick (F1 = 0.49). As illustrated in Figure 6a, SVM often confuses active events with the no-event class, due to its inability to encode temporal dependencies from sequential data.

Taken together, these results indicate that the OS-CNN model effectively balances performance across both frequent and less frequent event classes, while avoiding the sharp drop-offs observed in simpler or over-parameterized models.

4.4.2. Confusion Matrix Analysis: Imbalanced Dataset

This section presents a comprehensive evaluation of the proposed OS-CNN model and compares it with the other baseline models—SVM, GRU, and Transformer—using a balanced training set and an imbalanced test set. The evaluation focused on both confusion matrices and per-class evaluation metrics.

As shown in Figure 7, the SVM model (Figure 7a) exhibited poor discriminative ability under class distribution mismatch. Although the no-event class (class 0) is the majority in the test set, SVM misclassified a large portion of those samples as event classes, such as 48,747 as goal kick and 62,467 as free kick, among others, leading to only 90,216 correct predictions for class 0. This indicates the difficulty of the model in learning a robust boundary under cross-distribution generalizations, resulting in a degraded performance across all classes.

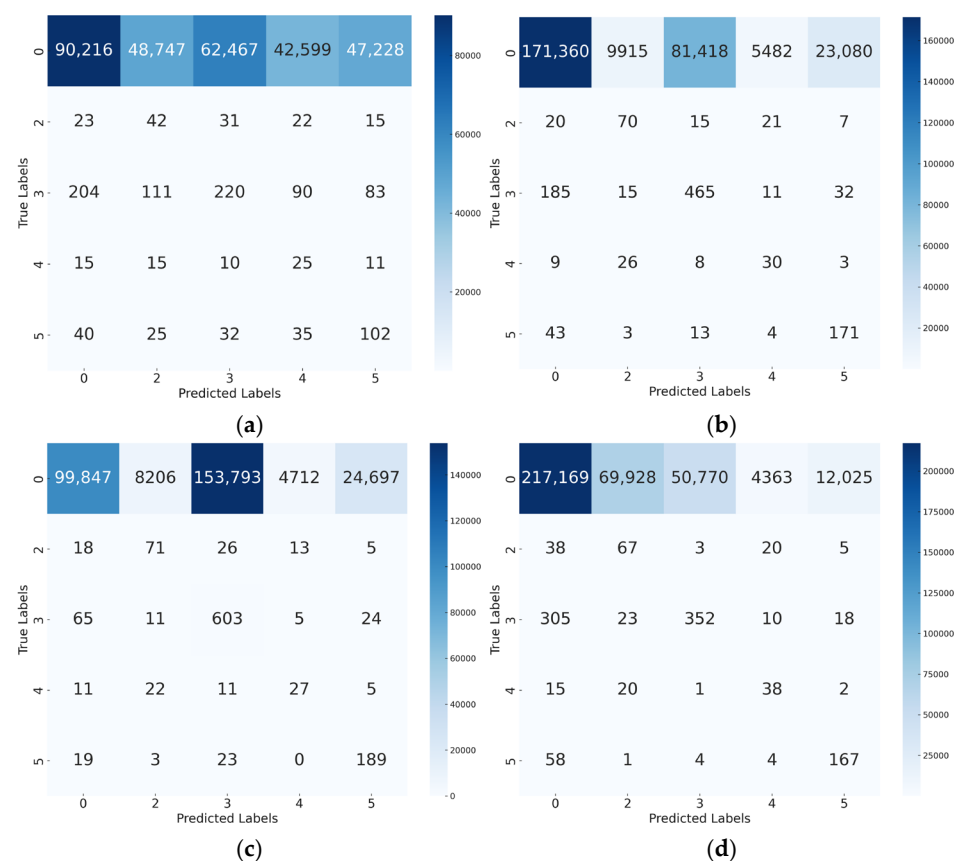


Figure 7. Confusion matrices for event prediction on the imbalanced dataset: (a) confusion matrix for SVM (Imbalanced dataset); (b) confusion matrix for GRU (Imbalanced dataset); (c) confusion matrix for Transformer (Imbalanced dataset); (d) confusion matrix for the OS-CNN (Imbalanced dataset).

The GRU and Transformer models demonstrated relatively improved robustness compared to SVM. The Transformer model showed higher recall for rare classes—e.g., achieving 85.17% recall for free kick and 80.77% for throw-in. Nevertheless, they still suffered from low precision and F1-scores due to frequent misclassification between event types and the dominant class. Notably, the GRU model showed more balanced recall but lower precision overall.

By contrast, the proposed OS-CNN model (Figure 7d) significantly outperformed all baselines under this setting. It correctly predicted 217,169 samples of the no-event class and captured rare events with superior accuracy: 67 goal kicks, 352 free kicks, 38 corner kicks, and 167 throw-ins. This superior discriminative ability translated into the highest per-class performance across most categories. As presented in Table 9, the OS-CNN model achieved the best F1-scores for both the event classes (1.87–2.68%) and the no-event class (85.36%), outperforming all other models.

Table 9. Per-Class Evaluation Metrics (Imbalanced Dataset).

Method	Goal Kick (2) (%)			Free Kick (3) (%)			Corner Kick (4) (%)			Throw-In (5) (%)			No-Event (0) (%)		
	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑	P ↑	R ↑	F1 ↑
Kościółek [7]	0.09	31.58	0.17	0.35	31.07	0.69	0.06	32.89	0.12	0.22	43.59	0.43	99.69	30.97	47.26
GRU [45]	0.70	52.63	1.38	0.57	65.68	1.13	0.54	39.47	1.07	0.73	73.08	1.45	99.85	58.84	74.04
Transformer [46]	0.85	53.38	1.68	0.39	85.17	0.78	0.57	35.53	1.12	0.76	80.77	1.50	99.89	34.28	51.04
Proposed Approach	0.95	50.38	1.87	0.69	49.72	1.36	0.86	50.00	1.68	1.37	71.37	2.68	99.81	74.56	85.36

Note: Precision (P), Recall (R), and F1-score (F1). Bold values indicate the best performance in each column.

These outcomes highlight the strength of OS-CNN in learning robust temporal representations that generalize well from balanced to imbalanced settings. While other deep learning models partially captured minority class boundaries, only OS-CNN maintained a reliable precision–recall tradeoff across all categories. This suggests that OS-CNN is highly suitable for real-time sports commentary systems, offering both accurate event recognition and stability in the face of data distribution shifts.

4.4.3. Loss Analysis for Balanced and Imbalanced Dataset

As shown in Table 10, in the Balanced setting, GRU achieved the lowest loss value (0.6046), followed closely by Transformer (0.6381) and OS-CNN (0.6658), while SVM recorded the highest loss (1.1738). This trend aligns with the overall F1-score results in Table 11, where GRU slightly outperformed OS-CNN and Transformer under balanced data conditions.

Table 10. Comparison of loss values for different models under Balanced and Imbalanced dataset settings.

Dataset	Method	Model	Loss
Balanced	Kościółek [7]	SVM	1.1738
	Base Model [45,46]	GRU	0.6046
		Transformer	0.6381
	Proposed Approach	OS-CNN	0.6658
Imbalanced	Kościółek [7]	SVM	2.3568
	Base Model [45,46]	GRU	1.9026
		Transformer	2.1953
	Proposed Approach	OS-CNN	0.2596

In the Imbalanced setting, the proposed OS-CNN model demonstrated a substantial advantage with the lowest loss value (0.2596), significantly lower than GRU (1.9026), Transformer (2.1953), and SVM (2.3568). This notable reduction in loss indicates that OS-CNN can effectively learn from minority-class instances while minimizing overfitting to the dominant “no-event” class, which is a common challenge in highly skewed distributions. These results further confirm OS-CNN’s robustness and superior generalization ability in real-world scenarios where event distributions are inherently imbalanced.

Table 11. Event Prediction Results (Balanced vs. Imbalanced).

Dataset	Method	Model	Recall $\uparrow$	Precision $\uparrow$	F1-Score $\uparrow$
Balanced	Kořciółek [7]	SVM	0.5286	0.5260	0.5271
	Base Model [45,46]	GRU Transformer	0.7539 0.7359	0.7562 0.7354	0.7547 0.7344
	Proposed Approach	OS-CNN	0.7531	0.7515	0.7470
Imbalanced	Kořciółek [7]	SVM	0.3099	0.9930	0.4708
	Base Model [45,46]	GRU Transformer	0.5886 0.3445	0.9946 0.9950	0.7376 0.5085
	Proposed Approach	OS-CNN	0.7448	0.9942	0.8503

Note: Bold values indicate the best performance in each column.

4.4.4. Results Summary

As presented in Table 11, in the Balanced setting, the GRU model achieved the highest F1-score (0.7547), slightly outperforming OS-CNN (0.7470). This result can be attributed to GRU's gated recurrent structure, which is effective in capturing long-range temporal dependencies and benefits from the evenly distributed event classes during both training and testing. Nevertheless, OS-CNN maintained a strong balance between recall (0.7531) and precision (0.7515), while clearly outperforming SVM (0.5271) and Transformer (0.7344). These results indicate that the proposed OS-CNN model, through its multi-scale convolutional design, can also stably extract key temporal features under balanced data distributions.

In the Imbalanced setting, the training dataset was balanced through the over-sampling of minority events, whereas the test set retained its original skewed event distribution, reflecting real-world soccer match conditions [7], where imbalanced datasets have been shown to better represent actual gameplay scenarios than artificially balanced datasets.

Under these circumstances, OS-CNN achieved high precision (0.9942) and a recall of 0.7448, resulting in an F1-score of 0.8503—substantially outperforming SVM (0.4708), GRU (0.7376), and Transformer (0.5085). These results demonstrate that OS-CNN is able to effectively learn minority-class patterns during training and to maintain strong generalization capability when evaluated on highly imbalanced test data. This robustness in recognizing rare but critical events underscores the suitability of OS-CNN for deployment in real-time commentary systems for actual soccer matches.

4.5. Commentary Generation with Prompt Engineering

The impact of prompt design on the expressiveness and response speed of the generated commentary was further investigated. The baseline prompts in [6,18] use template filling structures. While simple to implement, they lead to repetition and a mechanical tone.

The generated outputs from these templates exhibit significant redundancy (e.g., “Real Madrid has been awarded a free kick...”) and long generation time (>4 s), which impairs real-time usability. Table 12 presents a set of reference prompts used for game commentary generation in [6,18], categorized by event type. These prompts serve as structured templates to guide language models in producing concise and contextually appropriate outputs during the different phases of a match.

Improved Prompt Design

To address the limitations of rigidity and latency found in earlier prompt templates, a revised format was employed to ensure more dynamic and context-aware commentary generation. The updated prompts are designed with structured placeholders (e.g., {timestep}, {player}, {team}, {chosen_emotion}, {event_type}, {final_score}), which are dynamically populated based on real-time game metadata.

Table 12. Reference Prompts for Game Commentary Generation.

Event Type	Prompt
Opening Commentary	You are a football commentator, who is getting information about a football match at Santiago Bernabeu between Real Madrid and Manchester United. I will give you info about the match, and you will write commentary. Be truthful and concise, max two sentences per turn. Only add information about teams, players, or goals that I have specified. You may use as much of my input as required in the output, use what you need to sound natural. Avoid repeating yourself too often. Do not mention the commentary itself.
Goal Event	In [time] minute, there is a goal for [team], scored by [player], the actual score is [score].
Corner Kick Event	In [time] minute, the ball is kicked on corner by [player], [team] will have a chance.
Free Kick Event	In [time] minute, there is a free kick for [team] due to a foul on [player].
Throw-in Event	In [time] minute, the ball was kicked out from the field by [player] from [team].
Goal Kick Event	In [time] minute, the ball is out of play, and a goal kick is awarded to [team].
Match Summary	It is the end of the match, the score is [score], summarize the match.

Two distinct prompt types were created: one for goal events and the other for non-goal events. The goal event prompt directs the language model to describe how a scored goal affects match momentum, to explicitly reference the updated score, and to convey the appropriate emotional tone (e.g., excited, calm, or nervous) within a 75-word limit. The non-goal prompt focuses on sequences such as goal kicks, free kicks, corner kicks, and throw-ins. It highlights shifts in match atmosphere without mentioning goals, scores, or player errors, while maintaining expressive and emotionally conditioned language.

Table 13 list the prompt templates with field-level guidance, and Appendix A provides the complete set of prompt variants, including emotion- and score-dependent substitutions, to ensure transparency and reproducibility in commentary generation.

Table 13. Improved Commentary Generation Prompt.

Event Type	Prompt
Goal Event	At minute {timestep}, {player} from {team} scored a goal in the match against {benefiting_team}. Write a vivid and expressive commentary using a {chosen_emotion} tone. Clearly describe how this goal shifts the momentum or atmosphere of the match without exaggeration or subjective judgment. Mention the updated score: {final_score}. Keep the output under 75 words.
Non-goal Event	At minute {timestep}, {player} from {team} was involved in a sequence that led to a {event_type} awarded to {benefiting_team}. Write a vivid and expressive commentary using a {chosen_emotion} tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.

4.6. Commentary Evaluation Results

Although both [6,18] adopted template-based approaches for commentary generation, the latter introduced VLM by incorporating in-game images captured at the moment of the event. However, study [18] demonstrated that the overall error rate of the VLM is higher than that of a pure language model (LLM), and the generated commentary tends to be overly simplified and less informative. Based on this observation, ref. [6] was selected as the baseline method for comparison.

The effectiveness of the proposed commentary generation framework was assessed through experiments on 100 simulated soccer matches, with outcomes compared against a baseline method [6]. This evaluation focuses on two main aspects: linguistic quality

and generational efficiency. A range of natural language evaluation metrics was applied, including Distinct-1, Distinct-2, Lexical Diversity, Self-BLEU, and generation time, to assess the quality and diversity of the generated commentary. The results are summarized in Table 14. An example of an entire conversation during a match is provided in Appendix B.

Table 14. Comparison of Linguistic Quality and Generation Efficiency.

Method	Perplexity ↑	Distinct-1 ↑	Distinct-2 ↑	Lexical Diversity ↑	Self-BLEU ↑	Time (s) ↑
Czaplicki [6]	17.36	0.29	0.56	0.29	0.53	3.89
Proposed Approach	52.95	0.45	0.74	0.45	0.39	3.08

Note: Bold values indicate the best performance in each column.

Our method shows substantial improvements in both textual diversity and efficiency. Specifically, the Distinct-1 score improved from 0.29 to 0.45 and the Distinct-2 score improved from 0.56 to 0.74, indicating more diverse word choices and reduced repetition in the generated commentary. Lexical diversity also increased from 0.29 to 0.45, suggesting that the system produced more varied and expressive sentences.

In addition, the Self-BLEU score dropped from 0.53 to 0.39, confirming that our model generates more diverse outputs across different samples, which is especially important in real-time commentary settings where events of the same type may occur repeatedly.

Although the perplexity value increased from 17.36 to 52.95, this did not necessarily reflect a degraded fluency. In the context of sports commentary, higher perplexity often corresponds to more spontaneous, emotionally rich, and idiomatic expressions, rather than a rigid, template-based language.

Finally, the average generation time was reduced from 3.89 s to 3.08 s per sentence, demonstrating improved responsiveness. This latency reduction confirms the practicality of the system for live applications where quick and context-aware commentary generation is essential.

In summary, the proposed framework not only improves linguistic richness and reduces redundancy, but enhances the generation speed, offering a viable and scalable solution for real-time sports commentary systems.

Despite gains in commentary diversity and responsiveness, LLMs inevitably introduce risks, such as factual hallucinations, contextual inconsistency, and biased expressions. These issues are especially critical in real-time applications, where accuracy and appropriateness are essential. Although the current study lacks explicit risk-mitigation modules, future work could integrate lightweight rule-based post-filters to validate factual elements in the generated output (e.g., team name, event type), or apply consistency scoring modules to detect and suppress hallucinated content. In addition, prompt-level constraints and diversity-aware reranking strategies could help reduce biased language and repetition. Such measures provide practical avenues to enhance the trustworthiness and robustness of LLM-based commentary systems.

Furthermore, ethical considerations must not be overlooked when deploying automated commentary systems. Although our study uses simulated environments without real-world players or teams, future deployments may involve the use of identifiable names and match contexts. In such cases, proper consent and licensing must be secured to prevent the misuse or violations of personal rights. Additionally, while this technology is designed to complement rather than replace human commentators, there remains a possibility of workforce displacement in specific domains. Finally, as with all generative AI systems, these commentary tools could be repurposed to produce misleading or deepfake style narratives. Addressing these concerns through transparency, access control, and rigorous validation will be critical for responsible deployment.

To further evaluate the user perception of commentary correctness and narrative quality, a user study was conducted involving ten football game enthusiasts. Participants were shown a full match with commentary generated by four models—SVM (template-based), GRU, Transformer, and OS-CNN (all prompt-based). Each in-game event (e.g., throw-in, free kick, goal kick) was accompanied by four different commentaries, one from each model. While the primary goal was to assess the overall quality of the OS-CNN commentary, the GRU and Transformer outputs generated using the same prompt schema were also included to isolate the effect of event prediction accuracy on user preference. This design allowed us to assess whether OS-CNN’s improved prediction leads to more contextually appropriate commentary, even when prompt templates are shared.

Participants can choose one or more commentary texts that they think are correct, fluent, and natural. For each event, the vote rate for a given model was calculated as the number of participants selecting that model’s commentary / total participants (10) $\times$ 100%. This vote rate is used as an indicator of the percentage of human evaluators who judged the commentary as ‘Appropriate’, reflecting the degree to which each output aligns with human expectations regarding quality and accuracy. As summarized in Table 15, OS-CNN received the highest number of votes across all events, with 5 out of 10 events achieving full agreement (10/10). Transformer occasionally received moderate support, especially for free kick and goal kick scenarios, but never outperformed OS-CNN. GRU and SVM were consistently less favored. These results confirm that users not only prefer commentary that is linguistically diverse, but that aligns closely with actual gameplay—underscoring the value of accurate real-time event prediction.

Table 15. Human evaluation results comparing commentary preferences across four models (SVM, GRU, Transformer, and OS-CNN) for ten in-game events.

Event No.	Event Type	Percentage of Human Votes Marking as ‘Appropriate’			
		OS-CNN	Transformer	GRU	SVM
1	Throw-in (16 min)	100%	30%	10%	0%
2	Free kick (19 min)	70%	60%	30%	10%
3	Goal kick (34 min)	60%	40%	40%	0%
4	Goal kick (45 min)	100%	40%	10%	0%
5	Free kick (65 min)	100%	10%	20%	10%
6	Free kick (68 min)	80%	50%	0%	10%
7	Corner kick (71 min)	100%	0%	0%	0%
8	Free kick (75 min)	90%	30%	10%	0%
9	Free kick (81 min)	100%	30%	10%	0%
10	Throw-in (88 min)	50%	20%	10%	0%
Average		85%	31%	14%	3%

Note: Each cell represents the number of users (out of 10) who selected a given model’s commentary as satisfactory. OS-CNN consistently received the highest number of votes, demonstrating the effectiveness of combining event prediction with prompt-engineered generation. Numbers in parentheses denote the match minute. Bold values indicate the best performance in each column.

4.7. Latency Analysis for Real-Time Commentary Generation

To assess whether the proposed system meets real-time requirements, the end-to-end latency was decomposed into three components: data preprocessing, event prediction, and commentary generation. Table 16 presents the average processing time per sample for each stage, based on empirical measurements.

Table 16. End-to-End Latency Evaluation for Real-Time Feasibility.

Component	Description	Time per Sample (s or ms)
Data Processing	Data loading, formatting, feature scaling	0.131 s
	GRU inference	0.1987 ms
Event Prediction	Transformer inference	0.5195 ms
	OS-CNN inference	0.1909 ms
LLM Commentary Generation	LLaMA-based text generation	3.080 s
Total Latency	Preprocessing + OS-CNN + LLM	3.156 s

The data preprocessing stage—which involves raw input loading, formatting, and z-score normalization—incurs a latency of 0.131 s per sample. For the event prediction component, three models were evaluated for comparative purposes: GRU (0.1987 ms), Transformer (0.5195 ms), and OS-CNN (0.1909 ms). Among them, OS-CNN achieved the lowest latency, validating its suitability for real-time inference.

The LLM-based commentary generation module, implemented with a lightweight LLaMA decoder, represents the primary latency bottleneck. On average, commentary generation takes 3.080 s per sentence, which is significantly higher than other stages.

By aggregating all stages—data preprocessing (0.131 s), OS-CNN inference (0.1909 ms), and LLaMA generation (3.080 s)—the total end-to-end latency of the system is approximately 3.156 s. This total was empirically measured rather than estimated. The result demonstrates that the system can generate commentary t seconds ahead of the actual event ($t + 3$ scenario) with sufficient time margin, confirming its feasibility for real-time deployment.

5. Conclusions and Future Work

This study introduces a novel three-stage framework for real-time automatic commentary generation in soccer games, combining a time-series event prediction model with an LLM. Unlike conventional machine-learning approaches, our method leverages the temporal structure and dynamic characteristics of gameplay data to enhance both prediction accuracy and responsiveness.

Specifically, an OS-CNN model was employed to classify time-series sequences and anticipate key events, such as free kicks, corner kicks, and throw-ins. These predicted events were then used to construct dynamic prompts, which were processed using the LLaMA 3.3 model to generate natural and context-aware commentary in real time. The application of prompt engineering led to improvements in the diversity, fluency, and generation efficiency of the commentary output.

Overall, the proposed OS-CNN framework demonstrated superior performance across event prediction, commentary generation quality, and user preference evaluations. In event prediction (Table 11), it achieved an F1-score of 0.7470 in the Balanced setting—comparable to the best-performing GRU model (0.7547) and higher than SVM (0.5271) and Transformer (0.7344)—and attained the highest F1-score of 0.8503 in the more realistic Imbalanced setting, substantially exceeding all baselines while maintaining a strong balance between recall (0.7448) and precision (0.9942). In commentary generation (Table 14), the proposed approach improved diversity (Distinct-1: 0.45, Distinct-2: 0.74, Lexical Diversity: 0.45) and reduced phrase repetition (Self-BLEU: 0.39 vs. 0.53) compared to the template-based baseline, while also reducing generation time from 3.89 s to 3.08 s. Human evaluation results (Table 15) further confirmed its effectiveness, with OS-CNN receiving the highest vote rates across ten in-game events, often achieving unanimous preference, highlighting its ability to produce accurate, diverse, and engaging commentary suitable for real-time deployment. Beyond its technical contributions, the proposed system offers considerable real-world applicability. It can be deployed in live sports broadcasts and e-sports streams

to deliver automated, multilingual, and context-aware commentary in real time, reducing the dependence on human commentators and reducing production costs.

Despite these promising results, our study has several limitations. First, the performance of the model on minority event classes remained suboptimal owing to data imbalances. Some rare events are either missed or incorrectly predicted. Second, the evaluation of commentary quality relied primarily on qualitative analysis and lacked objective and quantifiable benchmarks. To address these issues, future work will explore advanced techniques, such as transfer learning, meta-learning, and few-shot learning, to enhance the model's generalization to rare-event classes.

Additionally, to broaden applicability, future research will explore its deployment across multiple real-time game environments beyond soccer. Although the current study focuses on the Google Research Football environment, the modular, domain-agnostic architecture can adapt to other game genres, such as basketball, racing, or multiplayer online battle arena (MOBA) games. This extension would facilitate comparative studies across diverse gameplay dynamics and evaluate the generalizability of both the event prediction and the language generation components.

In addition to these task-specific limitations, it is essential to acknowledge the broader limitations of deep learning-based methods. These models are often highly data-dependent and require extensive labeled datasets to achieve a reliable performance, which can be challenging to acquire in specialized domains, such as sports gameplay. Moreover, their black-box nature limits interpretability, making it difficult to understand or to debug incorrect predictions and generated content. Deep learning models may also struggle to generalize across different game environments, especially when the underlying rules or visual dynamics differ. Finally, the computational cost of training and deploying large-scale models can be prohibitive, specifically in low-latency or resource-constrained settings. Evaluating the performance of the proposed system under various computational budgets and real-time constraints across different environments will thus be a critical direction for future work. Addressing these limitations will require ongoing exploration of hybrid architecture, interpretable learning mechanisms, and efficient model compression techniques.

Author Contributions: Conceptualization, X.S. and K.C.; methodology, X.S. and A.Y.; software, X.S.; validation, X.S., M.Z. and G.A.; writing—original draft preparation, X.S.; writing—review and editing, J.P., A.Y. and K.C.; visualization, X.S.; supervision, K.C.; project administration, K.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) through the Artificial Intelligence Convergence Innovation Human Resources Development Program (grant number IITP-2025-RS-2023-00254592, 30%). This research was also supported by JOYCITY through research funding in 2024 for the project “Freestyle Game Artificial Intelligence Development” (40%). This research was additionally supported by the IITP (Institute of Information & Communications Technology Planning & Evaluation)-ICAN (ICT Challenge and Advanced Network of HRD) program funded by the Korean government (MSIT) (grant number IITP-2025-RS-2023-00260248, 30%).

Data Availability Statement: The source code and model checkpoints are not publicly available due to institutional and commercial constraints. However, we provide detailed pseudocode describing the data preparation process to support transparency and reproducibility. Requests for further details may be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

LLM	Large Language Models
SVM	Support Vector Machine
ANN	Artificial Neural Network
OS-CNN	Omni-Scale Convolutional Neural Network
Seq2Seq	Sequence-to-Sequence
LSTM	Long Short-Term Memory
LLaMA 3.3	Large Language Model Meta AI 3.3
GRF	Google Research Football
GPT-3.5	Generative Pre-trained Transformer 3.5
ChatGPT	Chat Generative Pre-trained Transformer
API	Application Programming Interface
CNN	Convolutional Neural Network
BART	Bidirectional and Auto-Regressive Transformer
EfficientDet	Efficient Object Detection
HUD	Heads-Up Display
FPS	First-Person Shooter
LLaVA v1.5	Large Language and Vision Assistant version 1.5
TCN	Temporal Convolutional Network
RNN	Recurrent Neural Network
GRU	Gated Recurrent Unit
XGBoost	eXtreme Gradient Boosting
MCC	Matthews Correlation Coefficient
AUC	Area Under the Curve
VLM	Vision Language Model
TTR	Type/Token Ratio
SMOTE	Synthetic Minority Over-sampling Technique

Appendix A

To ensure reproducibility and transparency, the complete set of prompt templates used for commentary generation is presented. The prompts are categorized into four stages: match introduction, goal event commentary, non-goal event commentary, and match summary (outro). All templates employ structured placeholders that are dynamically filled using real-time match data.

Table A1. Match Introduction Prompt.

Match Introduction
You are a football commentator covering a match at the Santiago Bernabeu between the home side, Real Madrid, and the visiting team, Manchester United. I will provide you with match information, and you will generate concise and vivid commentary. Use a maximum of two sentences per turn. Only mention players, teams, or events I have specified, and do not invent any new details. Avoid repetition, and do not refer to the commentary itself. This is the beginning of the match. Write a vivid opening line to set the scene, clearly stating that Real Madrid is the home team and Manchester United is the away team.

Table A2. Goal Event Prompt Template.

Goal Event
At minute {timestep}, {player} from {team} scored a goal in the match against {benefiting_team}. Write a vivid and expressive commentary using a {chosen_emotion} tone. Clearly describe how this goal shifts the momentum or atmosphere of the match without exaggeration or subjective judgment. Mention the updated score: {final_score}. Keep the output under 75 words.

Table A3. Non-Goal Event Prompt Template.

Non-Goal Event
At minute {timestep}, {player} from {team} was involved in a sequence that led to a {event_type} awarded to {benefiting_team}. Write a vivid and expressive commentary using a {chosen_emotion} tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.

Table A4. Examples with Variable Substitution.

Timestep	Team	Player	Event	Score	Emotion	Substituted Prompt
'	Real Madrid	10	goal kick	1-0	excited	At minute 25, No. 10 from Real Madrid was involved in a sequence that led to a goal kick awarded to Manchester United. Write a vivid and expressive commentary using an excited tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
12	Real Madrid	5	goal kick	1-1	calm	At minute 12, No. 5 from Real Madrid was involved in a sequence that led to a goal kick awarded to Manchester United. Write a vivid and expressive commentary using a calm tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
19	Real Madrid	8	goal kick	0-1	nervous	At minute 19, No. 8 from Real Madrid was involved in a sequence that led to a goal kick awarded to Manchester United. Write a vivid and expressive commentary using a nervous tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
26	Real Madrid	4	free kick	1-0	excited	At minute 26, No. 4 from Real Madrid was involved in a sequence that led to a free kick awarded to Manchester United. Write a vivid and expressive commentary using an excited tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
33	Real Madrid	5	free kick	1-1	calm	At minute 33, No. 5 from Real Madrid was involved in a sequence that led to a free kick awarded to Manchester United. Write a vivid and expressive commentary using a calm tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
40	Real Madrid	7	free kick	0-1	nervous	At minute 40, No. 7 from Real Madrid was involved in a sequence that led to a free kick awarded to Manchester United. Write a vivid and expressive commentary using a nervous tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.

Table A4. Cont.

Timestep	Team	Player	Event	Score	Emotion	Substituted Prompt
47	Real Madrid	4	corner kick	1-0	excited	At minute 47, No. 4 from Real Madrid was involved in a sequence that led to a corner kick awarded to Manchester United. Write a vivid and expressive commentary using an excited tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
54	Real Madrid	3	corner kick	1-1	calm	At minute 54, No. 3 from Real Madrid was involved in a sequence that led to a corner kick awarded to Manchester United. Write a vivid and expressive commentary using a calm tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
61	Real Madrid	5	corner kick	0-1	nervous	At minute 61, No. 5 from Real Madrid was involved in a sequence that led to a corner kick awarded to Manchester United. Write a vivid and expressive commentary using a nervous tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
68	Real Madrid	7	throw-in	1-0	excited	At minute 68, No. 7 from Real Madrid was involved in a sequence that led to a throw-in awarded to Manchester United. Write a vivid and expressive commentary using an excited tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
75	Real Madrid	8	throw-in	1-1	calm	At minute 75, No. 8 from Real Madrid was involved in a sequence that led to a throw-in awarded to Manchester United. Write a vivid and expressive commentary using a calm tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.
82	Real Madrid	10	throw-in	0-1	nervous	At minute 82, No. 10 from Real Madrid was involved in a sequence that led to a throw-in awarded to Manchester United. Write a vivid and expressive commentary using a nervous tone. Describe how this shift impacts the match's momentum or atmosphere, without explicitly mentioning any scoring or direct error. Keep the output under 75 words.

Table A5. Match Summary Prompt.

Match Summary
It is the end of the match at the Santiago Bernabéu between the home team, Real Madrid, and the visiting side, Manchester United. The final score is [final_score]. Summarize the key moments of the match in 1–2 natural sentences. Do not invent any events. Only refer to those I have provided. Avoid subjective opinions, emotional evaluations, or unnecessary praise. Focus on factual closure that matches the tone of a professional sports broadcast.

Appendix B

Table A6. Compared commentary results between [6] and our method.

Time (min)	Event	Latency (s)	Generated Commentary
3	Throw-in	3.85	[Czaplicki [6]] The match at Santiago Bernabeu is underway and Real Madrid's Player 8 has already made his presence felt, kicking the ball out of bounds just 3 min in. Manchester United will now have a throw-in opportunity to regain possession of the ball.
		3.35	[Proposed Approach] Pandemonium erupts in the 3rd minute as Player 8's desperate lunge sparks chaos! Manchester United pounces, sensing vulnerability, and wins a crucial throw-in, igniting a frenzied counter-attack!
16	Free kick	3.42	[Czaplicki [6]] Real Madrid has been awarded a free kick here at the Santiago Bernabeu in the 16th minute after a foul was committed on their player, number 10. The Manchester United defence will need to be cautious as Real Madrid looks to capitalize on this scoring opportunity.
		2.69	[Proposed Approach] Tension boils over as Manchester United's defense falters! A burst of pressure sparks chaos, and suddenly Real Madrid seize control, sensing blood in the water. The crowd erupts!
22	Goal	4.18	[Czaplicki [6]] Real Madrid have taken an early lead here at the Santiago Bernabeu, with Player 10 scoring the opening goal in the 22nd minute to put the home team up 1-0 against Manchester United. The crowd is electric as the Spanish giants take control of the match, putting pressure on the visiting Red Devils to respond.
		3.58	[Proposed Approach] UNBELIEVABLE! Player 10 unleashes a rocket of a shot, curves it in with precision! The Santiago Bernabéu ERUPTS! Real Madrid takes the lead, 1-0, in the 22nd minute!
26	Goal kick	3.53	[Czaplicki [6]] The referee has blown his whistle and signaled for a goal kick, which will be taken by Real Madrid, allowing them to regain possession of the ball from their own end of the Santiago Bernabeu pitch. Manchester United's players are already positioning themselves in anticipation of the restart, looking to put pressure on the home team.
		2.70	[Proposed Approach] Tension boils over! A sudden lapse sparks chaos, and Real Madrid seize the initiative with a goal kick! The Bernabéu faithful sense opportunity, as momentum swings dramatically in their favor!
33	Throw-in	3.66	[Czaplicki [6]] The crowd here at Santiago Bernabeu is waiting anxiously for the game to resume as Manchester United's player has cleared the ball, momentarily relieving pressure on their defense. Real Madrid will now have a throw-in, looking to regain possession and continue their attack.
		3.08	[Proposed Approach] Tension boils over as Manchester United seizes momentum! A flash of uncertainty sparks among Real Madrid's defenders, and suddenly the visitors are on the front foot, sensing opportunity. The throw-in is theirs, fueling a surge of energy that electrifies the Bernabéu air.

Appendix C

Table A7. Pseudocode of the GRF Event Dataset Construction Pipeline (code not released; pseudocode provided for reproducibility purposes).

Data Preparation Pseudocode	
Game Event Data Preparation Pipeline	
Input: Raw GRF match logs under folders episode_done_*, each file stores per-frame states.	
Output: Two .ts datasets: D_dim (dimension $\times$ time = 123×3), D_flat (flattened [123×3]); labels $Y \in \{0,2,3,4,5\}$ (0 = no-event).	
1: for each file f in episode_done_* do	
2: $S \leftarrow \text{ParseFrames}(\text{RemoveWrappers}(\text{Read}(f)))$	$\triangleright$ list of per-frame dictionaries
3: $E \leftarrow \{t \mid \text{game_mode}_{[t-1]} = 0 \wedge \text{game_mode}_t \in \{2,3,4,5\}\}$	$\triangleright$ event indices
4: for each $t \in E$ do	
5: if $t < 5$ then continue	
6: $W \leftarrow [t-5, t-4, t-3]$	$\triangleright$ 3 preceding frames, stride = 1
7: $x \leftarrow \text{ExtractFeatures}(S, W)$	$\triangleright$ ball state; team/player kinematics; $\triangleright$ yellow cards; tiredness; score; game_mode
8: $x \leftarrow \text{ZScoreNormalize}(x)$	$\triangleright$ across all time steps
9: Append(D_dim, ToDimTime(x ; 123×3))	
10: Append(D_flat, Flatten(x))	$\triangleright$ [123×3]-length vector
11: Append(Y , game_mode_t)	$\triangleright$ label = current game_mode
12: end for	
13: $R \leftarrow \text{RandomSample}(\text{NonEventIndices}(S) \setminus E, E)$	$\triangleright$ balancing with no-event samples
14: for each $r \in R$ do	
15: if $r < 5$ then continue	
16: $W \leftarrow [r-5, r-4, r-3]$; $x \leftarrow \text{ExtractFeatures}(S, W)$; $x \leftarrow \text{ZScoreNormalize}(x)$	
17: Append(D_dim, ToDimTime(x)); Append(D_flat, Flatten(x)); Append(Y , 0)	
18: end for	
19: end for	
20: WriteTS(D_dim, headers); WriteTS(D_flat, headers); LogParseFailures()	
21: return D_dim, D_flat, Y	

References

- Li, L.; Uttarapong, J.; Freeman, G.; Wohn, D.Y. Spontaneous, yet studios: Esports commentators' live performance and self-presentation practices. *Proc. ACM Hum.-Comput. Interact.* **2020**, *4*, 103. [\[CrossRef\]](#)
- Baughman, A.; Morales, E.; Agarwal, R.; Akay, G.; Feris, R.; Johnson, T.; Hammer, S.; Karlinsky, L. Large scale generative AI text applied to sports and music. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Barcelona, Spain, 25–29 August 2024; pp. 4784–4792.
- Ma, S.; Cui, L.; Dai, D.; Wei, F.; Sun, X. Livebot: Generating live video comments based on visual and textual contexts. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 27 January–1 February 2019; Volume 33, pp. 6810–6817.
- Rao, J.; Wu, H.; Liu, C.; Wang, Y.; Xie, W. MatchTime: Towards automatic soccer game commentary generation. *arXiv* **2024**, arXiv:2406.18530. [\[CrossRef\]](#)
- Xu, J.H.; Cai, Y.; Fang, Z.; Paliyawan, P. Promoting mental well-being for audiences in a live-streaming game by highlight-based bullet comments. *arXiv* **2021**, arXiv:2108.08083.
- Czaplicki, M. Live Commentary in a Football Video Game Generated by an AI. Mater's Thesis, University of Twente, Enschede, The Netherlands, 2023.
- Kościołek, J. Enhancing live commentary generation in soccer video games through event prediction with machine learning methods. In Proceedings of the TSCIT 41, Enschede, The Netherlands, 5 July 2024; pp. 1–8.
- Vatsal, S.; Dubey, H. A survey of prompt engineering methods in large language models for different NLP tasks. *arXiv* **2024**, arXiv:2407.12994. [\[CrossRef\]](#)
- Nimpattanavong, C.; Taveekitworachai, P.; Khan, I.; Nguyen, T.V.; Thawonmas, R.; Choensawat, W.; Sookhanaphibarn, K. Am I fighting well? Fighting game commentary generation with ChatGPT. In Proceedings of the IAIT '23: Proceedings of the 13th International Conference on Advances in Information Technology, Bangkok, Thailand, 6–9 December 2023; pp. 1–7.
- Renella, N.; Eger, M. Towards automated video game commentary using generative AI. In Proceedings of the AIIDE Workshop on Experimental Artificial Intelligence in Games, Salt Lake City, UT, USA, 8 October 2023; pp. 1–9.

11. Wang, Z.; Yoshinaga, N. From eSports data to game commentary: Datasets, models, and evaluation metrics. In Proceedings of the DEIM Forum, Tokyo, Japan, 1–3 March 2021; pp. 1–6.
12. Li, C.; Gandhi, S.; Harrison, B. End-to-end Let's Play commentary generation using multi-modal video representations. In Proceedings of the 14th International Conference on the Foundations of Digital Games, San Luis Obispo, CA, USA, 26–30 August 2019; pp. 1–7. [\[CrossRef\]](#)
13. Kameko, H.; Mori, S.; Tsuruoka, Y. Learning a game commentary generator with grounded move expressions. In Proceedings of the 2015 IEEE Conference on Computational Intelligence and Games (CIG), Tainan, Taiwan, 31 August–2 September 2015; pp. 177–184.
14. Zang, H.; Yu, Z.; Wan, X. Automated chess commentator powered by neural chess engine. *arXiv* **2019**, arXiv:1909.10413. [\[CrossRef\]](#)
15. Ishigaki, T.; Topic, G.; Hamazono, Y.; Noji, H.; Kobayashi, I.; Miyao, Y.; Takamura, H. Generating racing game commentary from vision, language, and structured data. In Proceedings of the 14th International Conference on Natural Language Generation, Scotland, UK, 20–24 September 2021; pp. 103–113.
16. Mamoru, M.D.L.S.; Panditha, A.D.; Perera, W.A.S.S.J.; Ganegoda, G.U. Automated commentary generation based on FPS gameplay analysis. In Proceedings of the 7th International Conference on Information Technology Research (ICITR), Moratuwa, Sri Lanka, 7–9 December 2022; pp. 1–5. [\[CrossRef\]](#)
17. Ishigaki, T.; Topic, G.; Hamazono, Y.; Kobayashi, I.; Miyao, Y.; Takamura, H. Audio commentary system for real-time racing game play. In Proceedings of the 16th International Natural Language Generation Conference: System Demonstrations, Prague, Czech, 11–15 September 2023; pp. 9–10.
18. Stournaras, G. Generating Automatic Commentary in Video Games Using Large Language and Vision Language Models. Master's Thesis, University of Twente, Enschede, The Netherlands, 2024.
19. Kurach, K.; Raichuk, A.; Stańczyk, P.; Zajac, M.; Bachem, O.; Espeholt, L.; Riquelme, C.; Vincent, D.; Michalski, M.; Bousquet, O.; et al. Google Research Football: A novel reinforcement learning environment. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 4501–4510.
20. Schmidhuber, J.; Gagliolo, M.; Wierstra, D. Evolino for recurrent support vector machines. *arXiv* **2005**, arXiv:cs/0512062. [\[CrossRef\]](#)
21. Wunderlich, T. Gradient-Based Learning and Regularization in Spiking Neurons. Ph.D. Thesis, Technische Universität Berlin, Berlin, Germany, 2024.
22. Kharoubi, R.; Mkhadri, A.; Oualkacha, K. High-dimensional penalized Bernstein support vector machines. *arXiv* **2023**, arXiv:2303.09066. [\[CrossRef\]](#)
23. Yan, W. Toward automatic time-series forecasting using neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2012**, *23*, 1028–1039.
24. Muñoz-Zavala, A.E.; Macías-Díaz, J.E.; Alba-Cuéllar, D.; Guerrero-Díaz-de-León, J.A. A literature review on some trends in artificial neural networks for modeling and simulation with time series. *Algorithms* **2024**, *17*, 76. [\[CrossRef\]](#)
25. Wang, Y.; Wu, H.; Dong, J.; Liu, Y.; Long, M.; Wang, J. Deep time series models: A comprehensive survey and benchmark. *arXiv* **2024**, arXiv:2407.13278. [\[CrossRef\]](#)
26. Petneházi, G. Recurrent neural networks for time series forecasting. *arXiv* **2019**, arXiv:1901.00069. [\[CrossRef\]](#)
27. Shi, J.; Jain, M.; Narasimhan, G. Time series forecasting (TSF) using various deep learning models. *arXiv* **2022**, arXiv:2204.11115. [\[CrossRef\]](#)
28. Bai, S.; Kolter, J.Z.; Koltun, V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv* **2018**, arXiv:1803.01271. [\[CrossRef\]](#)
29. Wu, H.; Hu, T.; Liu, Y.; Zhou, H.; Wang, J.; Long, M. TimesNet: Temporal 2D-variation modeling for general time series analysis. *arXiv* **2022**, arXiv:2210.02186.
30. Liu, Y.; Wu, H.; Wang, J.; Long, M. Non-stationary transformers: Exploring the stationarity in time series forecasting. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 9881–9893.
31. Dai, M.; Yuan, J.; Liu, H.; Wang, J. TSCF: An improved deep forest model for time series classification. *Neural Process. Lett.* **2024**, *56*, 13. [\[CrossRef\]](#)
32. Chen, X.; Qiu, P.; Zhu, W.; Li, H.; Wang, H.; Sotiras, A.; Wang, Y.; Razi, A. TimeMIL: Advancing multivariate time series classification via a time-aware multiple instance learning. *arXiv* **2024**, arXiv:2405.03140.
33. Tang, W.; Long, G.; Liu, L.; Zhou, T.; Blumenstein, M.; Jiang, J. Omni-Scale CNNs: A simple and effective kernel size configuration for time series classification. *arXiv* **2020**, arXiv:2002.10061.
34. Narin, A. Performance comparison of balanced and unbalanced cancer datasets using pre-trained convolutional neural network. *arXiv* **2020**, arXiv:2012.05585. [\[CrossRef\]](#)
35. Wei, Q.; Dunbrack, R.L., Jr. The role of balanced training and testing data sets for binary classifiers in bioinformatics. *PLoS ONE* **2013**, *8*, e67863. [\[CrossRef\]](#)
36. Velarde, G.; Weichert, M.; Deshmunkh, A.; Deshmane, S.; Sudhir, A.; Sharma, K.; Joshi, V. Tree boosting methods for balanced and imbalanced classification and their robustness over time in risk assessment. *Intell. Syst. Appl.* **2024**, *22*, 200354. [\[CrossRef\]](#)

37. Aguilar-Ruiz, J.S.; Michalak, M. Classification performance assessment for imbalanced multiclass data. *Sci. Rep.* **2024**, *14*, 10759. [[CrossRef](#)]
38. Ustuner, M.; Sanli, F.B.; Abdikan, S. Balanced vs imbalanced training data: Classifying RapidEye data with support vector machines. *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.* **2016**, *41*, 379–384. [[CrossRef](#)]
39. Sen, P. Speech disfluencies occur at higher perplexities. In Proceedings of the Workshop on the Cognitive Aspects of the Lexicon, Online, 12 December 2020; pp. 92–97.
40. Baevski, A.; Auli, M. Adaptive input representations for neural language modeling. *arXiv* **2018**, arXiv:1809.10853.
41. Toral, A.; Pecina, P.; Wang, L.; van Genabith, J. Linguistically-augmented perplexity-based data selection for language models. *Comput. Speech Lang.* **2015**, *32*, 11–26. [[CrossRef](#)]
42. Li, J.; Galley, M.; Brockett, C.; Gao, J.; Dolan, B. A diversity-promoting objective function for neural conversation models. *arXiv* **2015**, arXiv:1510.03055.
43. Lu, X. The relationship of lexical richness to the quality of ESL learners' oral narratives. *Mod. Lang. J.* **2012**, *96*, 190–208. [[CrossRef](#)]
44. Zhu, Y.; Lu, S.; Zheng, L.; Guo, J.; Zhang, W.; Wang, J.; Yu, Y. Tegygen: A benchmarking platform for text generation models. In Proceedings of the 41st International ACM SIGIR Conference on Research & Development in Information Retrieval, Ann Arbor, MI, USA, 8–12 July 2018; pp. 1097–1100.
45. Cho, K.; Van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; Bengio, Y. Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation. *arXiv* **2014**, arXiv:1406.1078.
46. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is All You Need. In Proceedings of the 30th Conference on Neural Information Processing Systems (NeurIPS), Long Beach, CA, USA, 4–9 December 2017.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.